

Predikcija kašnjenja letova na temelju podataka iz skladišta

Nadal, Gabriel

Undergraduate thesis / Završni rad

2025

Degree Grantor / Ustanova koja je dodijelila akademski / stručni stupanj: **University of Pula / Sveučilište Jurja Dobrile u Puli**

Permanent link / Trajna poveznica: <https://urn.nsk.hr/urn:nbn:hr:137:366766>

Rights / Prava: [In copyright](#) / [Zaštićeno autorskim pravom.](#)

Download date / Datum preuzimanja: **2025-10-15**



Repository / Repozitorij:

[Digital Repository Juraj Dobrila University of Pula](#)



SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

Gabriel Nadal

Predikcija kašnjenja letova na temelju podataka iz skladišta

ZAVRŠNI RAD

Pula, rujan, 2025. godine

SVEUČILIŠTE JURJA DOBRILE U PULI

FAKULTET INFORMATIKE

Gabriel Nadal

Predikcija kašnjenja letova na temelju podataka iz skladišta

ZAVRŠNI RAD

JMBAG: 0303110630 /redoviti student

Studijski smjer: Informatika

Kolegij: Skladišta i rudarenje podataka

Znanstveno područje: Društvene znanosti

Znanstveno polje: Informacijske znanosti

Znanstvena grana: Informacijski sustavi i informatologija

Mentor: izv. prof. dr. sc. Goran Oreški

Pula, rujan, 2025. godine

Sažetak

Ovaj završni rad bavi se izradom predikcijskog modela za procjenu kašnjenja zračnih letova korištenjem sustava poslovne inteligencije i skladišta podataka. Na temelju javno dostupnog skupa podataka o letovima njujorškog područja (JFK, LGA, EWR; 2013.) provedeni su ključni koraci: prikupljanje i čišćenje podataka, implementacija relacijskog i dimenzijskog modela, izrada ETL procesa, analiza i vizualizacija podataka te razvoj prediktivnog modela.

Relacijski model izrađen je u MySQL bazi, dok je dimenzijski model oblikovan u obliku zvjezdaste sheme. Transformacija i učitavanje podataka provedeni su u Apache Sparku, a vizualizacije su izrađene u Power BI-ju kroz interaktivne nadzorne ploče s mogućnostima slice/dice, drill-down/roll-up i pivot analize. Za predikciju kašnjenja razvijen je model temeljen na algoritmu Random Forest. Nakon početne verzije, model je evaluiran i poboljšán optimizacijom hiperparametara i prilagodbom praga odluke prema F2-mjeri radi veće osjetljivosti na stvarna kašnjenja.

Rad pokazuje kako primjena poslovne inteligencije i strojnog učenja omogućuje pretvaranje velikih operativnih zapisa u pravodobne i primjenjive uvide, čime se unapređuje planiranje resursa i točnost reda letenja.

Ključne riječi: poslovna inteligencija, skladište podataka, ETL proces, analiza podataka, Power BI, strojno učenje, Random Forest, predikcija kašnjenja letova

Abstract

This bachelor's thesis presents the development of a predictive model for estimating flight delays using business intelligence (BI) systems and a data warehouse. Based on a publicly available dataset of flights in the New York area (JFK, LGA, EWR; 2013), the key steps included data collection and cleaning, implementation of relational and dimensional models, design of an ETL process, data analysis and visualization, and the creation of a predictive model.

The relational model was implemented in a MySQL database, while the dimensional model was built as a star schema. Data transformation and loading were carried out using Apache Spark, and interactive dashboards were created in Power BI, providing slice/dice, drill-down/roll-up, and pivot analyses. For delay prediction, a Random Forest algorithm was applied. After the initial version, the model was evaluated and improved through hyperparameter tuning and by adjusting the decision threshold according to the F2-score to increase sensitivity to actual delays.

This work demonstrates how the integration of business intelligence and machine learning can turn large volumes of operational flight data into timely, actionable insights, supporting better resource planning and improving schedule accuracy.

Keywords: business intelligence, data warehouse, ETL process, data analysis, Power BI, machine learning, Random Forest, flight delay prediction

Sadržaj

1. Uvod.....	1
2. Otkrivanje i analiza dataseta.....	2
2.1. Odabir dataseta.....	2
2.2. Opis dataseta	3
2.3. Analiza dataseta	3
3. Dizajn i implementacija relacijskog modela.....	8
3.1. Predprocesiranje podataka	8
3.2. Analiza zahtjeva i poslovnih pravila.....	10
3.3. Konceptualni model (ER dijagram)	11
3.4. Relacijski model baze podataka.....	12
3.5. Implementacija baze u MySQL-u	14
4. Dimenzijski model podataka	17
5. ETL proces	19
5.1. Extract	19
5.2. Transform.....	21
5.3. Load.....	25
5.4. Pregled ključnih tablica	26
6. Vizualizacija i poslovna analiza	28
6.1. Slice analiza	28
6.2. Dice analiza	31
6.3. Roll-up i Drill-down analiza	32
6.4. Pivot analiza	34
7. Razvoj predikcijskog modela.....	36
7.1. Priprema i odabir podataka	36
7.2. Odabir algoritma	36
7.3. Hiperparametri.....	37
7.4. Postupak treniranja i praćenje učenja	38
7.5. Bazni model	38

7.6. Finalni predikcijski model.....	41
7.6.1. Konfiguracija modela	42
8. Rezultati i evaluacija modela	50
8.1. Klasifikacijske metrike	50
8.2. Matrica zabune	51
8.3. Optimizacija praga na F2.....	52
8.4. Globalne mjere diskriminativnosti.....	53
8.5. Važnost značajki.....	53
8.6. Predikcija na novim letovima	54
9. Zaključak	55

1. Uvod

U suvremenom donošenju odluka podaci su temelj za učinkovito planiranje i brze reakcije. Zračni promet pritom generira velike količine raznolikih zapisa od planiranih i stvarnih vremena polijetanja i dolaska do podataka o prijevozniku, ruti i udaljenosti pa je za pouzdanu analitiku najprije potrebno izgraditi čvrst i nadziran podatkovni temelj. Ovaj rad prikazuje cjelovit postupak, od uspostave skladišta podataka do razvoja prediktivnog modela kašnjenja leta, korištenjem javnog skupa podataka o letovima njujorškog područja (JFK, LGA, EWR; 2013.).

Prvi korak bio je izgraditi skladište podataka u kojem se sirovi zapisi prikupljaju, čiste, standardiziraju i povezuju u jedinstvenu cjelinu. Skladište uklanja nedosljednosti i duplikate, osigurava referencijalni integritet te omogućuje da svaki redak predstavlja jedan let. Takva organizacija čini podatke konzistentnima i provjerljivima te stvara stabilnu osnovu na kojoj se svi kasniji koraci mogu reproducirati i pouzdano provjeriti. Na temelju skladišta proveden je ETL proces koji podatke transformira u dimenzijski model prilagođen poslovnoj analizi i izvještavanju te oblikuje značajke dostupne prije polijetanja, poput mjeseca, dana u tjednu, sata planiranog polijetanja, identiteta prijevoznika, polazišnog i odredišnog aerodroma i kategorizirane udaljenosti rute. Na tako uređenim temeljima izrađen je prediktivni model kašnjenja leta, središnja točka ovoga rada. Model se temelji na algoritmu Random Forest, koji je prikladan za otkrivanje nelinearnih odnosa i složenih interakcija među vremenskim i operativnim značajkama. Trening se provodi na stratificiranoj podjeli podataka (80:20), a performanse se ocjenjuju standardnim metrikama poput točnosti, preciznosti, odziva, F1 i F2. Poseban naglasak stavljen je na F2-mjeru, koja daje veću težinu odzivu, jer je u praksi važnije pravodobno prepoznati stvarna kašnjenja nego izbjeći svaku lažnu uzbunu. Dodatno se podešava prag odluke kako bi se postigla optimalna ravnoteža između preciznosti i osjetljivosti.

Ovaj rad predstavlja praktični primjer razvoja sustava poslovne inteligencije, u ovom slučaju usmjerenog na analizu i predviđanje kašnjenja letova. Krajnji cilj, uz izgradnju skladišta podataka i izradu preglednih vizualizacija, bio je primijeniti strojno učenje na prikupljene podatke radi pravodobnog prepoznavanja mogućih kašnjenja. Stoga je razvijen prediktivni model temeljen na algoritmu Random Forest, kojim se nastoji unaprijediti planiranje i operativno odlučivanje u zračnom prometu.

2. Otkrivanje i analiza dataseta

U ovom poglavlju postavlja se temelj za prediktivno modeliranje. Objašnjava se izbor i izvor podataka, precizira se sastav dataseta i ključni atributi te način definiranja ciljne varijable kašnjenja. Provedena analiza razjašnjava operativni kontekst i ograničenja podataka te priprema okvir za konstrukciju značajki, odabir algoritma i strategiju treniranja, validacije i evaluacije.

2.1. Odabir dataseta

Za izgradnju modela odabran je javno dostupan skup podataka o letovima s platforme Kaggle (Flights), koji obuhvaća sve komercijalne operacije s aerodroma JFK, LGA i EWR tijekom 2013. godine. Dataset je domenski relevantan, vremenski konzistentan i dovoljno velik za pouzdanu evaluaciju, a istodobno operativno čist: svaka instanca predstavlja pojedini let s planiranim i ostvarenim vremenima, identifikatorima rute i prijevoznika te udaljenošću. Podaci odražavaju sezonalnost, zagušenje zračnog prostora i druge realne efekte, pa uočeni obrasci i trenirani model imaju smisla i izvan laboratorijskih uvjeta. Dostupnost u CSV formatu olakšava reproducibilno učitavanje, ETL i verzioniranje transformacija.

2.2. Opis dataseta

Dataset sadrži 261 877 zapisa i 29 atributa. Svaki zapis predstavlja jedan let s aerodroma JFK, LGA ili EWR u 2013. godini. Središte čine vremenski atributi koji razlikuju planirana i stvarna vremena polijetanja i dolaska, što omogućuje izračun kašnjenja na dolasku. Uz vremenske varijable prisutni su identifikatori prijevoznika te polazišnih i odredišnih aerodroma, kao i udaljenost rute u miljama, čime se mogu pratiti geografski i operativni obrasci. Vremenska dimenzija dodatno je izražena atributima mjeseca i dana u tjednu te planiranim vremenom polijetanja iz kojeg se izvodi sat polijetanja za analizu obrazaca po dobu dana. Zabilježen je i razlog odgode polaska, koristan za deskriptivno objašnjenje uzroka, uz napomenu da se u prediktivnoj postavci pažljivo tretira kako bi se izbjeglo curenje informacija.

2.3. Analiza dataseta

Kao početni korak izrađena je skripta koja automatizira osnovnu eksplorativnu analizu te pruža pregled strukture i kvalitete podataka, omogućujući rano prepoznavanje potencijalnih problema. Skripta učitava izvorni CSV uz UTF-8 kodiranje, prikazuje dimenzije skupa, tipove podataka i uzorni prikaz prvih redaka kako bi se provjerila konzistentnost zaglavlja i formata zapisa.

Na slici 1 se jasno vidi ispis nekoliko prvih redaka dataseta s pripadajućim nazivima stupaca. Ovaj ispis omogućuje uvid u točne nazive i poredak atributa, njihov tip (npr. numerički, tekstualni ili vremenski) te uobičajene vrijednosti. Uočljivo je da svaki redak predstavlja jedan let s polazišnim i odredišnim aerodromom, planiranim i stvarnim vremenima polijetanja i dolaska, identifikatorom prijevoznika, duljinom rute te eventualnim kašnjenjima izraženima u minutama. Prikaz služi kao prva kontrolna točka: potvrđuje da su zaglavlja ispravno učitana, da nema neželjenih znakova ni pomaknutih stupaca te da je format vremena dosljedan, što je ključno za sve kasnije faze modeliranja i analize.

```
import pandas as pd
import os

# Definirali smo relativnu putanju do datoteke
datoteka = os.path.join(os.path.dirname(__file__), "flights_main.csv")

# Učitali smo podatke iz CSV datoteke
data = pd.read_csv(datoteka, encoding="utf-8")

# Prikazali smo osnovne informacije o podacima
print(data.head())
print("Veličina skupa podataka:", data.shape)
print("Nazivi stupaca:", data.columns.tolist())

# Provjerili smo nedostajuće vrijednosti
print("Nedostajuće vrijednosti po stupcu:")
print(data.isna().sum())

# Ispisali smo jedinstvene vrijednosti za svaki stupac
print("Jedinstvene vrijednosti po stupcu:")
for column in data.columns:
    print(f"{column}: {data[column].unique()[:10]} ...")

# Provjerili smo tipove podataka
print(data.dtypes)

# Analizirali smo frekvencije vrijednosti po stupcima
print("Frekvencije vrijednosti po stupcu:")
for column in data.columns:
    print(f"{column}:")
    print(data[column].value_counts(), "\n")
```

Slika 1 Početna analiza dataseta

Slika 2 prikazuje rezultat provjere ukupnog broja redaka i stupaca u datasetu. U gornjem dijelu slike vidljiv je ispis funkcije shape, koja vraća vrijednost (261877, 29). Ovaj prikaz potvrđuje da skup podataka sadrži 261 877 zapisa (redaka), pri čemu svaki redak odgovara jednom letu, te 29 atributa (stupaca) koji bilježe različite vremenske, geografske i operativne informacije. Vizualni dokaz veličine skupa od ključne je važnosti jer pokazuje da obujam podataka omogućuje pouzdano treniranje i evaluaciju modela strojnog učenja, uz dovoljno primjera za segmentiranje po vremenskim razdobljima, aviokompanijama i drugim relevantnim kriterijima. Ovaj korak osigurava da su podaci reprezentativni, da nema nenamjernih gubitaka zapisa prilikom učitavanja te da se može planirati daljnja obrada i resursi potrebni za ETL i analitiku.

```
print("Veličina skupa podataka:", data.shape)
Veličina skupa podataka: (261877, 29)
```

Slika 2. Dimenzije skupa podataka

Slika 3 prikazuje pregled nedostajućih vrijednosti po stupcu nakon početnog učitavanja podataka. Na slici je vidljiv ispis koji za svaki atribut navodi broj i udio praznih zapisa, što omogućuje procjenu kvalitete izvora i rano donošenje odluke o strategiji obrade. Ovaj korak identificira stupce bez praznina koji se mogu izravno koristiti u analizi, kao i attribute s manjim udjelom nedostataka.

```
Nedostajuće vrijednosti po stupcu:
id                                0
year                              0
month                             0
day                               0
day_of_week                       0
dep_time                          0
sched_dep_time                    0
dep_delay_time                     0
reason_dep_delay                   0
arr_time                           0
sched_arr_time                     0
arr_delay_time                     0
reason_arr_delay                   0
carrier                           0
flight_num                         0
tailnum                           0
origin                            0
destination                       0
air_time                           0
distance                           0
hour                               0
minute                             0
airline_name                       0
departure_city                     0
departure_country                  0
departure_airport_name             0
destination_city                   0
destination_country                0
destination airport name           0
```

Slika 3. Nedostajućih vrijednosti po stupcu

Slika 4 sažeto prikazuje broj jedinstvenih vrijednosti po stupcima i time daje brzi uvid u kardinalnost atributa. Na temelju tog pregleda razlikuju se niskokardinalne varijable (npr. mjesec, dan u tjednu) od visokokardinalnih identifikatora (npr. let, aerodrom, prijevoznik), što pomaže pri normalizaciji u modelu i izboru načina kodiranja u pripremi značajki.

```
Jedinstvene vrijednosti po stupcu:
id: [ 0 1 2 3 4 5 7 9 10 11] ...
year: [2013] ...
month: [ 1 10 11 12 2 3 4 5 6 7] ...
day: [ 1 2 3 4 5 6 7 8 9 10] ...
day_of_week: ['Tuesday' 'Wednesday' 'Thursday' 'Friday' 'Saturday' 'Sunday' 'Monday'] ...
dep_time: ['05:17' '05:33' '05:42' '05:44' '05:54' '05:57' '05:58' '05:59' '06:00'
'06:02'] ...
sched_dep_time: ['05:15' '05:29' '05:40' '05:45' '06:00' '05:58' '05:59' '06:05' '06:10'
'06:15'] ...
dep_delay_time: [ 2. 4. -1. -6. -4. -3. -2. 0. 8. 11.] ...
reason_dep_delay: ['Slow Ground Operations' 'Ground Crew Coordination' 'Taxiing Delay'
'Early Departure' 'On Time' 'Baggage Handling Delay' 'Gate Change'
'Minor Technical Delays' 'De-icing Process' 'Air Traffic Congestion'] ...
arr_time: ['08:30' '08:50' '09:23' '10:04' '08:12' '07:40' '07:09' '07:53' '08:49'
'08:53'] ...
sched_arr_time: ['08:19' '08:30' '08:50' '10:22' '08:37' '07:28' '07:23' '07:45' '08:51'
'08:56'] ...
arr_delay_time: [ 11. 20. 33. -18. -25. 12. -14. 8. -2. -3.] ...
reason_arr_delay: ['Boarding Delays' 'Air Traffic Congestion' 'Early Arrival'
'Minor Technical Delays' 'Passenger Late Boarding'
'Last-Minute Paperwork' 'Late Arrival of Aircraft' 'Medical Emergency'
'Slow Ground Operations' 'Ground Equipment Issues'] ...
carrier: ['UA' 'AA' 'B6' 'DL' 'EV' 'MQ' 'US' 'WN' 'VX' 'FL'] ...
flight_num: [1545 1714 1141 725 461 1696 5708 301 49 71] ...
tailnum: ['N14228' 'N24211' 'N619AA' 'N804JB' 'N668DN' 'N39463' 'N829AS' 'N3ALAA'
'N793JB' 'N657JB'] ...
origin: ['EWR' 'LGA' 'JFK'] ...
destination: ['IAH' 'MIA' 'BQN' 'ATL' 'ORD' 'IAD' 'PBI' 'TPA' 'LAX' 'BOS'] ...
air_time: [227. 160. 183. 116. 150. 53. 138. 149. 158. 345.] ...
distance: [1400 1416 1089 1576 762 719 229 733 1028 1005] ...
hour: [ 5 6 7 8 18 9 10 11 12 13] ...
minute: [15 29 40 45 0 58 59 5 10 30] ...
airline_name: ['United Air Lines Inc.' 'American Airlines Inc.' 'JetBlue Airways']
```

Slika 4. Broj jedinstvenih vrijednosti po stupcu

3. Dizajn i implementacija relacijskog modela

Relacijski sloj predstavlja prvi strukturirani prikaz domene letova nakon sirovog CSV-a te služi kao stabilna osnova za dimenzijski model i analitiku. U ovom poglavlju opisuje se put od pripreme podataka do implementacije sheme i inicijalnog punjenja tablica, uz objašnjenje podjele skupa na 80:20 koja omogućuje istodobno oblikovanje modela i provjeru njegove robusnosti na novim zapisima.

3.1. Predprocesiranje podataka

Prije definiranja tablica i veza provedeno je predprocesiranje kako bi se izvorni zapisi doveli u dosljedan i modeliranju prikladan oblik. Ulazni CSV učitao je s UTF-8 kodiranjem; standardizirani su šifarnici prijevoznika i aerodroma, uklonjene ili ispravljene nelogičnosti, provjerena je konzistentnost rute i udaljenosti te je planirano vrijeme polijetanja pouzdano parsirano u minute od ponoći, iz čega je izveden sat polijetanja. Na temelju provjere jedinstvenosti odabran je prirodni ključ zapisa leta, a uvedeni su i nadomjesni ključevi radi jednostavnijeg referenciranja u kasnijim fazama. Skripta u nastavku reproducibilno izvodi sve korake i priprema podjelu skupa na 80:20.

Slika 5 prikazuje skriptu koja reproducibilno provodi cjelokupno predprocesiranje podataka. Skripta učitava izvorni CSV, standardizira šifrnike prijevoznika i aerodroma, parsira planirano vrijeme polijetanja u minute i izvodi sat polijetanja te provodi podjelu skupa u omjeru 80:20. Ovaj postupak osigurava dosljedan i kontroliran prijelaz sa sirovih podataka na strukturirani oblik spreman za modeliranje i kasniju evaluaciju.

```
CSV_FILE_PATH = "flights_main.csv" # Odredili smo putanju do CSV datoteke

df = pd.read_csv(CSV_FILE_PATH, delimiter=',') # Učitali smo podatke i ispisali dimenzije
print("CSV size before: ", df.shape)

# Standardizirali smo nazive zemalja
df['departure_country'] = df['departure_country'].replace('USA', 'United States')
df['destination_country'] = df['destination_country'].replace('USA', 'United States')
df['departure_country'] = df['departure_country'].replace('UK', 'United Kingdom')
df['destination_country'] = df['destination_country'].replace('UK', 'United Kingdom')

df = df.dropna() # Uklonili smo redove s nedostajućim vrijednostima

# Standardizacija naziva stupaca
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')

print("CSV size after: ", df.shape)
print(df.head())

duplicates = df.duplicated().sum() # Provjerili smo postojanje duplikata
print(f"Number of duplicates: {duplicates}")

if duplicates > 0: # Uklonili smo duplikate ako postoje
    df = df.drop_duplicates()
    print(f"CSV size after removing duplicates: {df.shape}")

# Podijelili smo podatke na 80:20
df20 = df.sample(frac=0.2, random_state=1)
df80 = df.drop(df20.index)

print("CSV size 80: ", df80.shape)
print("CSV size 20: ", df20.shape)

# Spreмили smo obrađene podatke u nove datoteke
df80.to_csv("flights_processed_80.csv", index=False)
df20.to_csv("flights_processed_20.csv", index=False)
```

Slika 5. Skripta za predprocesiranje skupa podataka i podjelu 80:20

Slika 6 prikazuje rezultat provedenog predprocesiranja i podjele skupa podataka u omjeru 80:20. Na njoj se vidi potvrda ukupnog broja zapisa, pri čemu je 209 502 redaka izdvojeno za treniranje, a 52 375 za testiranje. Ovaj prikaz služi kao provjera da su svi koraci čišćenja, standardizacije i podjele uspješno izvedeni te da su podskupovi jednako strukturirani i spremni za daljnju obradu.

```
[5 rows x 29 columns]
Number of duplicates: 0
CSV size 80: (209502, 29)
CSV size 20: (52375, 29)

Predprocesiranje završeno!
Stvorene datoteke:
- flights_processed_80.csv (80% podataka za transakcijski model)
- flights_processed_20.csv (20% podataka za ETL direktno iz CSV-a)
```

Slika 6. Rezultat predprocesiranja i podjele (80:20)

3.2. Analiza zahtjeva i poslovnih pravila

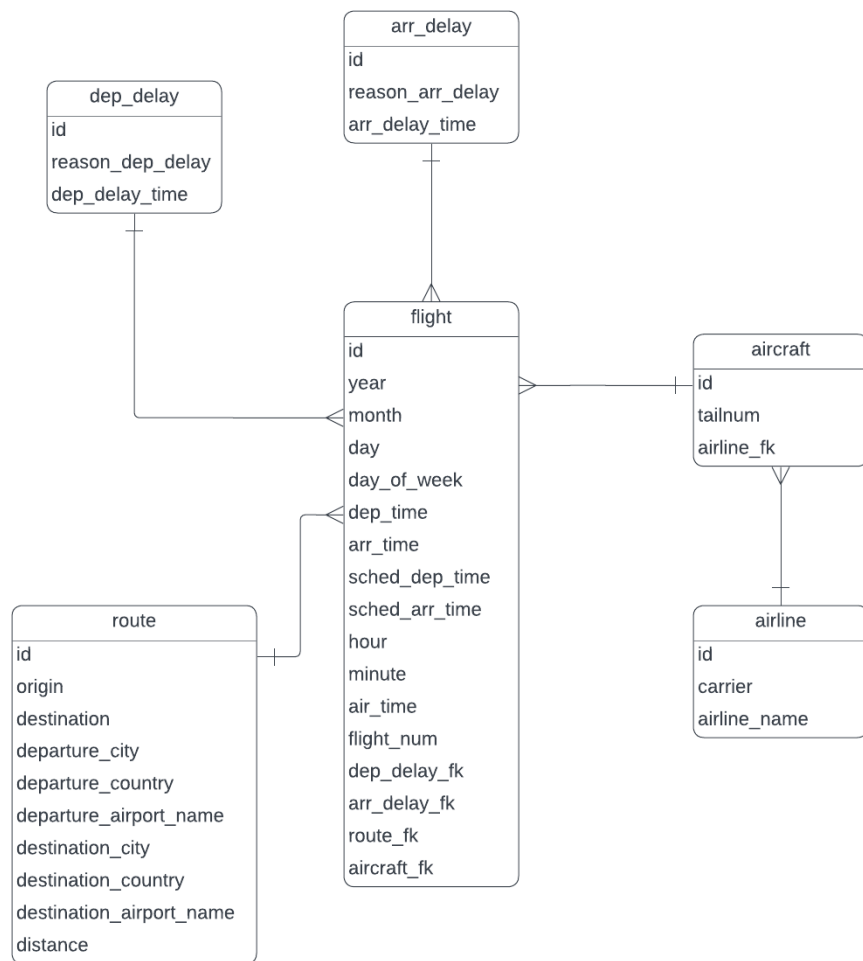
Nakon pregleda izvornog skupa oblikovani su funkcionalni i podatkovni zahtjevi koji usmjeravaju relacijski dizajn. Identificirani su ključni entiteti domene zračnog prometa: avioprijevoznik, zrakoplov, aerodrom, ruta, let i razlog kašnjenja. Svaki entitet nosi vlastite opisne attribute i sudjeluje u odnosima koji prate operativni tijek od plana do realizacije. Za let se zahtijeva jedinstvena identifikacija u kombinaciji s datumom, prijevoznikom i rutom, uz pohranu planiranih i ostvarenih vremena polijetanja i dolaska te pripadajućih kašnjenja izraženih u minutama. Poslovna pravila određuju način povezivanja entiteta i štite integritet: svaki let pripada točno jednom prijevozniku, polazi s jednog aerodroma i slijeće na drugi (polazište i odredište ne smiju biti jednaki), udaljenost je strogo pozitivna, a ruta se normalizira kako bi se izbjegla redundancija [3]. Kašnjenja se vode po fazi operacije (polazak, dolazak). Budući da uzroci kašnjenja mogu biti višestruki, uvodi se šifarnik razloga i povezna tablica koja omogućuje evidentiranje više stavki kašnjenja za isti let s fazom i trajanjem. Integritet je osiguran referencijalnim ograničenjima te

dodatnim pravilima poput provjere formata vremena, zabrane negativnih vrijednosti kašnjenja i jedinstvenosti kombinacije prijevoznik–broj leta–datum–polazište.

3.3. Konceptualni model (ER dijagram)

Konceptualni model obuhvaća šest ključnih entiteta: Airline, Aircraft, Route, Flight, DepDelay i ArrDelay. Središnji entitet Flight bilježi kalendarske podatke te planirana i ostvarena vremena polijetanja i dolaska. Airline definira prijevoznika, Aircraft evidentira zrakoplov i njegovu pripadnost, a Route sažima par polazište–odredište s pripadajućim opisima i udaljenosti. Kašnjenja su normalizirana u zasebne entitete DepDelay i ArrDelay, svaki s razlogom i trajanjem. Kardinalnosti su čitljive: jedan prijevoznik ima više zrakoplova, jedan zrakoplov i jedna ruta povezuju se s više letova, a let opcionalno referencira zapise o kašnjenjima. Dizajn slijedi 3NF: opisni podaci i šifrnici odvojeni su od događaja leta, veze su osigurane referencijalnim ograničenjima, a primarni ključevi su nadomjesni radi jednostavnog referenciranja uz jedinstvena ograničenja nad prirodnim ključevima. Takva organizacija daje čitljiv i održiv model koji jasno povezuje prijevoznika, zrakoplov, rutu i eventualna kašnjenja kroz jedinstveni zapis leta.

Slika 7 prikazuje konceptualni model u obliku ER dijagrama koji definira ključne entitete domene zračnog prometa i njihove međusobne odnose. Na dijagramu su jasno istaknute glavne tablice, poput Airline, Aircraft, Route i Flight, te poveznice s entitetima DepDelay i ArrDelay koji bilježe kašnjenja na polasku i dolasku. Ovaj prikaz služi kao temelj za izradu relacijskog modela, jer vizualno prikazuje strukturu podataka, kardinalnosti veza i pravila koja osiguravaju integritet baze podataka.

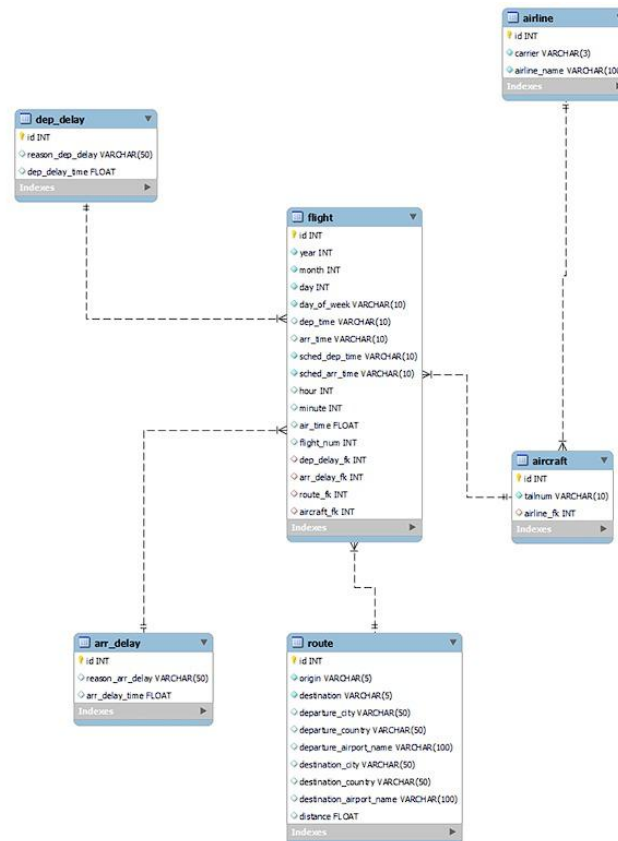


Slika 7. ER dijagram relacijskog modela

3.4. Relacijski model baze podataka

Nakon izrade konceptualnog modela definiran je relacijski model koji se izravno implementira u MySQL-u. Podaci su organizirani u jasno razdvojene tablice povezane primarnim i stranim ključevima, čime se osiguravaju konzistentnost, referencijalni integritet i pouzdana izvedba u operativnom okruženju. Svaka tablica ima jednoznačan primarni ključ, a veze među tablicama odražavaju poslovna pravila i podržavaju učinkovit ETL i tipične analitičke upite.

Slika 8 prikazuje sve tablice relacijskog modela s pripadajućim primarnim i stranim ključevima te međusobnim vezama. Vidljivo je da je flight središnja tablica koja referencira airline, aircraft i route, dok dep_delay i arr_delay standardiziraju razloge i trajanja kašnjenja. Dijagram potvrđuje primjenu treće normalne forme i jasno pokazuje kako se osigurava referencijalni integritet i učinkovit rad ETL-a.



Slika 8. Relacijski model baze podataka

Relacijski model obuhvaća šest glavnih tablica koje čine jezgru domene. Tablica **airline** pohranjuje identitet prijevoznika (ID, IATA kod i naziv). Tablica **aircraft** bilježi zrakoplove i njihovu pripadnost prijevozniku. Tablica **route** opisuje par polazište–odredište uz pripadajuće gradove, države, nazive aerodroma i udaljenost rute. Tablice **dep_delay** i **arr_delay** standardiziraju razloge i trajanja kašnjenja na polasku, odnosno dolasku. Tablica **flight** je središnji operativni zapis koji povezuje sve prethodne cjeline te sadrži kalendarske atribute, planirana i

ostvarena vremena, broj leta i strane ključeve prema ruti, zrakoplovu i šifarnicima kašnjenja. Sva su polja tipizirana u skladu s namjenom (vremenski i numerički tipovi za vremena i trajanja, ograničeni tekstualni tipovi za kodove), a primarni ključevi su surrogate cjelobrojni identifikatori radi jednostavnog referenciranja.

Kardinalnosti slijede logiku domene: **airline–aircraft** 1:N, **aircraft–flight** 1:N, **route–flight** 1:N. Više letova može referencirati isti zapis razloga kašnjenja na polasku ili dolasku (**dep_delay–flight** 1:N, **arr_delay–flight** 1:N). **flight** ostaje centralna točka koja povezuje prijevoznika, zrakoplov, rutu i moguća kašnjenja uz očuvanje integriteta preko stranih ključeva i odgovarajućih ograničenja.

Model slijedi 3NF: opisni podaci i šifarnici su u vlastitim tablicama, rute su deduplicirane, a razlozi kašnjenja standardizirani. Integritet osiguravaju PK/FK, jedinstvena ograničenja nad prirodnim kodovima (npr. IATA) i check ograničenja nad rasponima vrijednosti (npr. zabrana negativnih minuta kašnjenja). Indeksi su postavljeni na često filtrirane stupce (datumi u **flight**, IATA kodovi u **airline/route**) radi stabilne izvedbe ETL-a i analitičkih upita. Rezultat je čitljiv, održiv i performantan model koji pouzdano podupire daljnje faze skladišta i prediktivne analitike.

3.5. Implementacija baze u MySQL-u

Relacijski model implementiran je u MySQL sustavu za upravljanje bazama podataka kako bi se osiguralo stabilno i reproducibilno okruženje za ETL i analitiku. Postupak je odvijen u tri koraka. Najprije je definirana shema i pripremljene DDL definicije koje preciziraju tipove podataka, primarne ključeve i strane ključeve nad tablicama **airline**, **aircraft**, **route**, **dep_delay**, **arr_delay** i **flight**. Zatim su tablice kreirane u InnoDB mehanizmu s podrškom za transakcije i referencijalni integritet, uz postavljanje indeksa na često filtriranim atributima, poput datuma u tablici **flight** te IATA kodova u tablicama **airline** i **route** [8]. Na kraju je provedeno inicijalno punjenje iz pripremljenih CSV datoteka, čime je uspostavljen konzistentan „izvor istine“ spreman za daljnje transformacije i analizu.

Slika 9 prikazuje izvod DDL naredbi CREATE TABLE za ključne tablice, zajedno s ograničenjima i indeksima koji osiguravaju referencijalni integritet i učinkovito izvršavanje upita. Ovaj prikaz spaja konceptualni i relacijski dizajn s konkretnom implementacijom u MySQL-u.

```
# Putanja do predprocesirane CSV datoteke (80% podataka)
CSV_FILE_PATH = "flights_processed_80.csv"

# Učitavanje CSV datoteke u dataframe
df = pd.read_csv(CSV_FILE_PATH, delimiter=',')
print(f"CSV size: {df.shape}") # Print dataset size
print(df.head()) # Preview first few rows

# Database Connection
Base = declarative_base()

# Definiranje sheme baze podataka
# -----
class Airline(Base):
    __tablename__ = 'airline'
    id = Column(Integer, primary_key=True, autoincrement=True)
    carrier = Column(String(3), nullable=False, unique=True)
    airline_name = Column(String(100), nullable=False)

class Aircraft(Base):
    __tablename__ = 'aircraft'
    id = Column(Integer, primary_key=True, autoincrement=True)
    tailnum = Column(String(10), nullable=False, unique=True)
    airline_fk = Column(Integer, ForeignKey('airline.id'))

class Route(Base):
    __tablename__ = 'route'
    id = Column(Integer, primary_key=True, autoincrement=True)
    origin = Column(String(5), nullable=False)
    destination = Column(String(5), nullable=False)
    departure_city = Column(String(50))
    departure_country = Column(String(50))
    departure_airport_name = Column(String(100))
    destination_city = Column(String(50))
    destination_country = Column(String(50))
    destination_airport_name = Column(String(100))
    distance = Column(Integer)
```

Slika 9. Definiranje sheme baze podataka

Kreiranje i punjenje automatizirano je Python skriptom koja putem SQLAlchemy-ja uspostavlja vezu s MySQL-om te izvršava DDL i DML operacije. Nakon učitavanja izvornih CSV datoteka u DataFrame objekte provedene su validacije koje uključuju uklanjanje duplikata, provjeru tipova i raspona vrijednosti te usklađivanje šifrnika. Redoslijed umetanja prilagođen je ovisnostima

stranih ključeva kako bi se izbjegla kršenja integriteta: najprije su popunjeni šifrnici airline i route (te po potrebi aircraft), zatim su upisani zapisi o kašnjenjima u tablice dep_delay i arr_delay, a na kraju su učitani letovi u središnju tablicu flight koja referencira prethodno popunjene entitete. Umetanja su grupirana u transakcije kako bi se osigurala atomarnost i omogućio povrat stanja u slučaju pogreške. Po završetku su provjerene brojnosti po tablicama, testiran referencijalni integritet kako bi se potvrdilo da nema siročadi u odnosima stranih ključeva te su izvršeni probni upiti spajanja radi potvrde ispravnog povezivanja podataka.

Slika 10 prikazuje izrezak Python/SQLAlchemy skripte i pripadni log koji potvrđuje uspješno punjenje tablica airline i route, uz naznačene transakcijske korake i broj upisanih redaka. Ovaj prikaz dokumentira da ETL slijedi dogovoreni redoslijed umetanja i da se integritet podataka očuva tijekom procesa.

```
# **1. Umetanje aviokompanije**
airlines = df[['carrier', 'airline_name']].drop_duplicates() # Dohvatimo jedinstvene aviokompanije
airlines_list = airlines.to_dict(orient="records") # Pretvori u listu rječnika

session.bulk_insert_mappings(Airline, airlines_list) # Bulk insert
session.commit()

airline_map = {a.carrier: a.id for a in session.query(Airline).all()} # Stvori mapiranje aviokompanija
print(f"Uneseno {len(airlines_list)} aviokompanija")

# **2. Umetanje ruta**
routes = df[['origin', 'destination', 'departure_city', 'departure_country',
            'departure_airport_name', 'destination_city', 'destination_country',
            'destination_airport_name', 'distance']].drop_duplicates() # Dohvacamo jedinstvene rute

routes_list = routes.to_dict(orient="records") # Pretvori u listu rječnika

session.bulk_insert_mappings(Route, routes_list) # Bulk insert
session.commit()

# Kreiraj mapiranje ruta - kombinacija origin i destination
route_map = {}
for route in session.query(Route).all():
    key = f"{route.origin}_{route.destination}"
    route_map[key] = route.id

print(f"Uneseno {len(routes_list)} ruta")
```

Slika 10. Primjer punjenja tablica (airline, route)

4. Dimenzijski model podataka

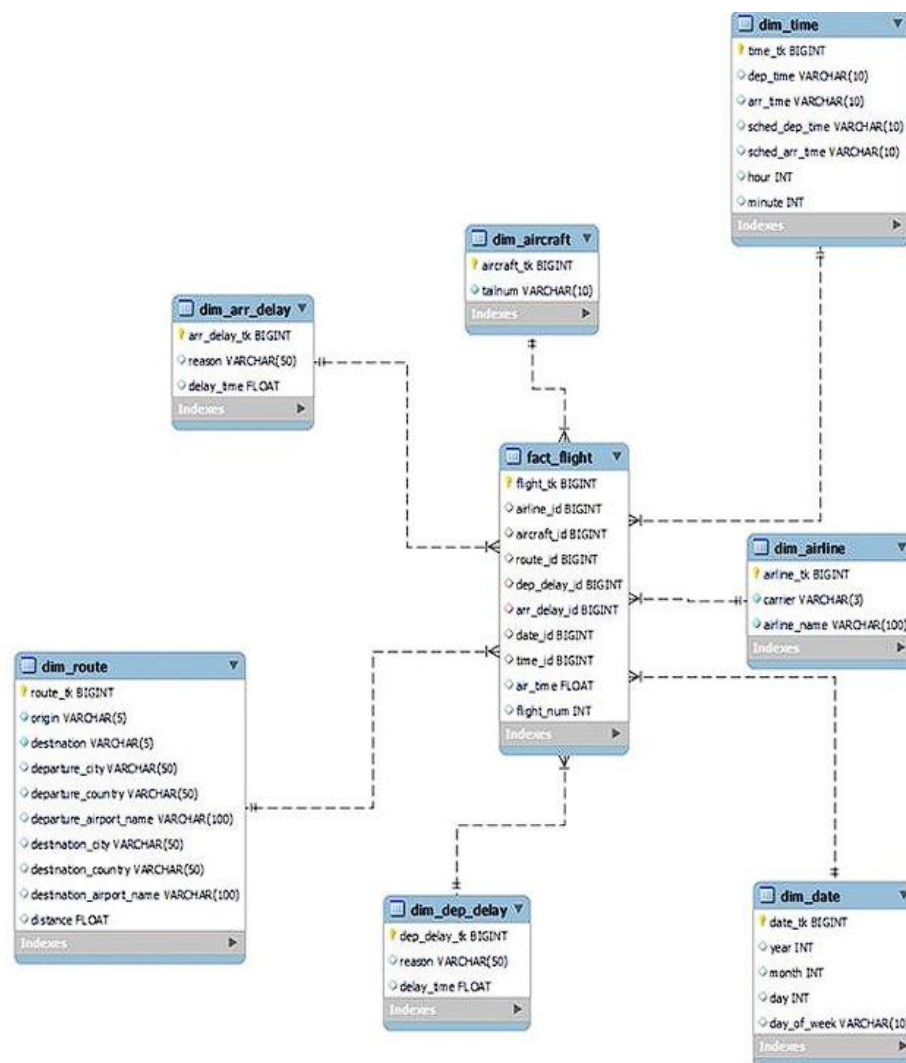
Dimenzijski model organizira podatke primarno za analizu i izvještavanje. Za razliku od relacijskog modela, koji je optimiran za transakcijsku konzistenciju i normalizaciju, dimenzijski pristup strukturira informacije oko poslovnog procesa i tipičnih analitičkih pitanja, pa su upiti brži, hijerarhije jasnije, a mjere lako agregabilne. U ovom slučaju cilj je omogućiti višedimenzionalni pogled na letne operacije kroz vrijeme, prijevoznika, rutu, zrakoplov i uzroke kašnjenja, pri čemu je cijela struktura prilagođena BI alatima i standardnim metrikama performansi [1].

U središtu modela nalazi se tablica **fact_flight** koja sadrži numeričke mjere poput vremena u zraku, kašnjenja na polasku i dolasku te udaljenosti, zajedno sa stranim ključevima prema pripadajućim dimenzijama. Dimenzijske tablice daju kontekst tim mjerama: one opisuju tko je let obavio, kada se let dogodio, između kojih aerodroma je let realiziran i kojim je zrakoplovom izveden, kao i koji je razlog eventualne odgode. Unutar dimenzija definirane su hijerarhije koje podržavaju uobičajene drill-down scenarije; vremenska dimenzija omogućuje analizu od godine preko kvartala i mjeseca do dana, a dnevna vremenska dimenzija dodatno razlikuje sate i minute te grupira zapise po dobu dana. Geografska hijerarhija u dimenziji rute vodi od države i grada do konkretnog aerodroma. Radi stabilnosti i jednostavne povezanosti s činjenicama, dimenzije koriste (surrogate) ključeve, dok se prirodni kodovi čuvaju kao opisna polja i za potrebe konsistentnog izvještavanja.

Model je oblikovan kao **star shema** (zvjezdasta shema) u kojoj je **fact_flight** središnja tablica, a sve dimenzije su na nju izravno vezane. Dimenzija **dim_airline** pohranjuje identitet prijevoznika kroz IATA kod i puni naziv. **dim_date** nosi kalendarske attribute poput godine, kvartala, mjeseca, dana i dana u tjednu, dok **dim_time** razrađuje unutar-dnevnu granularnost kroz sat, minutu i oznaku doba dana. **dim_route** opisuje polazišni i odredišni aerodrom zajedno s gradom, državom i nazivom aerodroma te omogućuje geografske presjeke i agregacije po pravcima. **dim_aircraft** identificira zrakoplov registracijom (tailnum) i, gdje je dostupno, dodatnim opisnim poljima. Uzroke odgoda standardiziraju dimenzije **dim_dep_delay** i **dim_arr_delay**; one sadrže razrede i opise razloga, dok se minute kašnjenja vode kao mjere u činjenicama kako bi se zadržala čista semantika i omogućile pravilne agregacije. Ovakva struktura pojednostavljuje spojeve (fact ↔ dim), daje predvidljive indekse, dobro se ponaša u BI okruženju i omogućuje konformnost zajedničkih dimenzija poput vremena i rute kroz sve izvještaje. Model je implementiran

programatski, koristeći SQLAlchemy u Pythonu, što osigurava ponovljivost izgradnje sheme i kontrolirano punjenje iz pripremljenih izvora podataka.

Slika 11 grafički prikazuje star shemu s tablicom činjenica `fact_flight` u središtu i dimenzijama vremena, prijevoznika, rute, zrakoplova te uzroka kašnjenja. Dijagram jasno ističe strani ključ svake dimenzije u tablici činjenica te mjere koje se agregiraju u analizama, što potvrđuje prilagođenost modela BI okruženju.



Slika 11. Dimenzijski model podataka

5. ETL proces

ETL (Extract, Transform, Load) predstavlja ključni korak u izgradnji skladišta podataka: podaci se preuzimaju iz izvora, obrađuju u konzistentan analitički oblik i učitavaju u ciljnu shemu. U ovom je projektu proces implementiran u Apache Sparku, što omogućuje obradu velikog broja zapisa o letovima uz reproducibilnost i dobre performanse.

5.1. Extract

Ekstrakcija je provedena iz dva izvora kako bi se zadržala podjela 80:20 i omogućila neovisna provjera procesa. CSV datoteka (20 %) učitava se Sparkom uz uključene zaglavlja i automatsko prepoznavanje sheme, pri čemu se odmah bilježe dimenzije skupa, praznine i mogući izdvojenici. Paralelno se iz transakcijskog MySQL modela (80%) dohvaćaju tablice airline, aircraft, route, dep_delay, arr_delay i flight preko JDBC konekcije s MySQL driverom. Time su oba izvora dostupna u obliku Spark DataFrameova i spremna za jedinstvenu obradu.

Slika 12 prikazuje dio Python skripte koja učitava CSV datoteku u Spark DataFrame. Na njoj se vidi čitanje datoteke s uključenim zaglavlјima, određivanje tipova podataka i inicijalno ispisivanje dimenzija skupa te broja potencijalno praznih zapisa. Ovaj prikaz potvrđuje da su podaci ispravno učitani i spremni za daljnje čišćenje.

```
from spark_session import get_spark_session

def extract_from_csv(file_path):

    spark = get_spark_session("ETL_Extract_CSV")

    print(f"Extracting data from CSV: {file_path}")

    df = spark.read \
        .option("header", True) \
        .option("inferSchema", True) \
        .csv(file_path)

    print(f"Extracted {df.count()} rows from CSV")

    return df
```

Slika 12. extract_csv.py

Slika 13 prikazuje skriptu za dohvaćanje podataka iz MySQL baze putem JDBC veze. Jasno se vide parametri konekcije, SQL upiti za odabir tablica i prijenos podataka u Spark DataFrame. Prikaz potvrđuje da je cjelokupna baza dostupna u analitičkom okruženju i da je moguće kombinirati zapise iz dva različita izvora.

```
from spark_session import get_spark_session

def extract_table(table_name):

    spark = get_spark_session("Flights_ETL_Extract_MySQL")

    jdbc_url = "jdbc:mysql://127.0.0.1:3306/flights_dw?useSSL=false"
    connection_properties = {
        "user": "root",
        "password": "root",
        "driver": "com.mysql.cj.jdbc.Driver"
    }

    print(f"Extracting table: {table_name}")
    df = spark.read.jdbc(url=jdbc_url, table=table_name, properties=connection_properties)
    print(f"Extracted {df.count()} rows from {table_name}")

    return df

def extract_all_tables():
    """
    Ekstrahuje sve tabele iz transakcijskog modela letova
    """
    print("Starting extraction of all MySQL tables...")
    tables = {
        "airline": extract_table("airline"),
        "aircraft": extract_table("aircraft"),
        "route": extract_table("route"),
        "dep_delay": extract_table("dep_delay"),
        "arr_delay": extract_table("arr_delay"),
        "flight": extract_table("flight")
    }
    print("Completed extraction of all MySQL tables")

    return tables
```

Slika 13. extract_csv.py

5.2. Transform

Transformacija usklađuje i objedinjuje izvore u konzistentan analitički model. Šifarnici prijevoznika, zrakoplova, ruta i razloga kašnjenja čiste se i standardiziraju (trim, tipiziranje, formatiranje naziva), uklanjaju se duplikati te se generiraju surogat ključevi pomoću determinističkog poretka i funkcije `row_number()` nad prozorom `Window.orderBy(...)`. Dimenzije

se, gdje je primjenjivo, obogaćuju 20-postotnim CSV uzorkom i ponovno deduplikiraju, a nepoznate vrijednosti mapiraju se na “UNKNOWN” kako bi veze ostale potpune. Tablica činjenica nastaje spajanjem obogaćenih letova s dimenzijama preko prirodnih ključeva (uz broadcast join radi performansi), pri čemu se izračunavaju izvedene mjere poput planiranog i stvarnog trajanja te oznake točnosti. Ugrađene su kontrole kvalitete koje prate udio zapisa s popunjenim obveznim ključevima i provjeravaju raspon vrijednosti. Kao reprezentativan primjer prikazuje se transformacija dimenzije ruta. Najprije se normaliziraju MySQL atributi i uklanjaju duplikati na uređenom paru (origin, destination), zatim se ako je prisutan, priprema i 20 % CSV izvor s istim atributima te se oba izvora spajaju metodom unionByName uz dodatnu deduplikaciju [7]. Time nastaje jedinstven, očišćen kandidat za šifrarnik ruta.

Slika 14 prikazuje Spark kod kojim se normaliziraju i standardiziraju atributi rute iz MySQL-a i CSV-a. Nakon uklanjanja duplikata oba izvora spajaju se metodom unionByName, a zatim se ponovno provjerava jedinstvenost zapisa. Na prikazu su vidljivi koraci koji osiguravaju da se podaci o rutama ujednače u jedinstven, potpuno očišćen skup.

```

from pyspark.sql.functions import col, trim, initcap, lit, row_number, current_timestamp, coalesce
from pyspark.sql.window import Window
from spark_session import get_spark_session

def transform_route_dim(mysql_route_df, csv_flights_df=None):

    spark = get_spark_session()
    print("Transforming route dimension...")

    # Normalizacija MySQL podataka
    mysql_df = (
        mysql_route_df
        .select(
            col("id").cast("long").alias("route_id"),
            trim(col("origin")).alias("origin"),
            trim(col("destination")).alias("destination"),
            initcap(trim(col("departure_city"))).alias("departure_city"),
            initcap(trim(col("departure_country"))).alias("departure_country"),
            initcap(trim(col("departure_airport_name"))).alias("departure_airport_name"),
            initcap(trim(col("destination_city"))).alias("destination_city"),
            initcap(trim(col("destination_country"))).alias("destination_country"),
            initcap(trim(col("destination_airport_name"))).alias("destination_airport_name"),
            col("distance").cast("double")
        )
        .dropDuplicates(["origin", "destination"])
    )

    # Normalizacija CSV podataka
    if csv_flights_df:
        csv_df = (
            csv_flights_df
            .select(
                trim(col("origin")).alias("origin"),
                trim(col("destination")).alias("destination"),
                initcap(trim(col("departure_city"))).alias("departure_city"),
                initcap(trim(col("departure_country"))).alias("departure_country"),

```

Slika 14. Normalizacija MySQL i CSV izvora, standardizacija i union bez duplikata

Nakon konsolidacije podataka dimenziji rute deterministički se dodjeljuje surogatni ključ `route_id` pomoću `Window.orderBy("origin","destination")` i `row_number()`. Na izlazu se projiciraju samo relevantni atributi (`origin`, `destination`, `grad`, `država`, `naziv aerodroma`, `distance`), čime nastaje čista dimenzijska tablica spremna za učitavanje.

Slika 15 prikazuje naredbe za dodjelu surogat ključa route_tk koristeći Window.orderBy("origin", "destination") i funkciju row_number(). U završnom dijelu koda jasno je istaknuto izdvajanje samo onih atributa koji su potrebni u dimenziji, što jamči da konačna tablica sadrži samo čiste i relevantne stupce.

```
        initcap(trim(col("departure_airport_name"))).alias("departure_airport_name"),
        initcap(trim(col("destination_city"))).alias("destination_city"),
        initcap(trim(col("destination_country"))).alias("destination_country"),
        initcap(trim(col("destination_airport_name"))).alias("destination_airport_name"),
        col("distance").cast("double")
    )
    .withColumn("route_id", lit(None).cast("long"))
    .dropDuplicates(["origin", "destination"])
)

# Kombinacija MySQL i CSV podataka
combined_df = mysql_df.unionByName(csv_df)
else:
    combined_df = mysql_df

# Brisanje duplikata
combined_df = combined_df.dropDuplicates(["origin", "destination"])

# Dodavanje surrogate ključeva
window = Window.orderBy("origin", "destination")

final_df = (
    combined_df
    .withColumn("route_tk", row_number().over(window))
    .select(
        "route_tk", "origin", "destination", "departure_city", "departure_country",
        "departure_airport_name", "destination_city", "destination_country",
        "destination_airport_name", "distance"
    ) # Brisanje SCD kolona
)

print(f"Created route dimension with {final_df.count()} records")
return final_df
```

Slika 15. Generiranje surogat ključa i formiranje izlazne sheme dimenzije

5.3. Load

Učitavanje završava proces kontroliranim upisom u MySQL dimenzijski model. Redoslijed poštuje ovisnosti: najprije se upisuju dimenzije (vrijeme, prijevoznik, zrakoplov, ruta, razlozi odgoda), a zatim `fact_flight`. Upis obavlja parametarska funkcija `DataFrame.write.jdbc(...)` u načinu `append`, uz transakcijsku potporu InnoDB-a i idempotentno ponašanje zahvaljujući deduplikaciji prije upisa i determinističkom generiranju ključeva. Nakon učitavanja provjeravaju se brojnosti, referencijalni integritet (odsutnost “siročadi”) i probni agregacijski upiti nad činjenicama, čime se potvrđuje da su podaci točni, usklađeni i spremni za analitiku.

Slika 16 prikazuje dio koda kojim se očišćeni `DataFrame` zapisa upisuje u dimenzijski model baze. Jasno su označeni parametri veze, ime ciljne tablice i način rada `append`, što osigurava stabilno punjenje i sprječava dupliciranje redaka.

```
from pyspark.sql import DataFrame

def write_spark_df_to_mysql(spark_df: DataFrame, table_name: str, mode: str = "append"):
    """
    Upisuje Spark DataFrame u MySQL star schema bazu
    """
    jdbc_url = "jdbc:mysql://127.0.0.1:3306/dimenzijski_model?useSSL=false" # star schema
    connection_properties = {
        "user": "root",
        "password": "root",
        "driver": "com.mysql.cj.jdbc.Driver"
    }

    print(f"Writing {spark_df.count()} rows to table `{table_name}` with mode `{mode}`...")
    print(f"Database URL: {jdbc_url}")

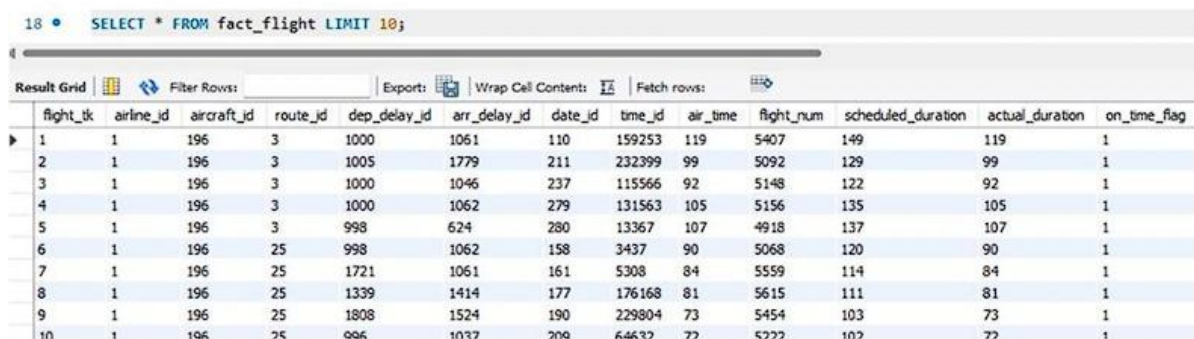
    try:
        spark_df.write.jdbc(
            url=jdbc_url,
            table=table_name,
            mode=mode,
            properties=connection_properties
        )
        print(f" Successfully wrote to `{table_name}`.")
    except Exception as e:
        print(f" Error writing to `{table_name}`: {str(e)}")
        raise
```

Slika 16. Primjer funkcije za učitavanje podataka

5.4. Pregled ključnih tablica

Središnja tablica činjenica **fact_flight** konsolidira kvantitativne mjere i strane ključeve prema svim dimenzijama. Uz tehnički ključ `flight_tk`, pohranjuje `airline_id`, `aircraft_id`, `route_id`, `dep_delay_id`, `arr_delay_id`, `date_id`, `time_id`, mjere poput `air_time` i `flight_num` te izvedene `scheduled_duration`, `actual_duration` i `on_time_flag`. Surogati se mapiraju u transformaciji, a referencijalni integritet validira se provjerama potpunosti i raspona.

Slika 17 prikazuje strukturu tablice `fact_flight` u MySQL bazi. Na slici se jasno uočavaju svi stupci s kvantitativnim mjerama, surogat ključ `flight_tk` i strani ključevi koji povezuju tablicu s pripadajućim dimenzijama. Ovaj prikaz potvrđuje da su sve veze ispravno uspostavljene i da se nad podacima mogu izvoditi brze agregacije.



flight_tk	airline_id	aircraft_id	route_id	dep_delay_id	arr_delay_id	date_id	time_id	air_time	flight_num	scheduled_duration	actual_duration	on_time_flag
1	1	196	3	1000	1061	110	159253	119	5407	149	119	1
2	1	196	3	1005	1779	211	232399	99	5092	129	99	1
3	1	196	3	1000	1046	237	115566	92	5148	122	92	1
4	1	196	3	1000	1062	279	131563	105	5156	135	105	1
5	1	196	3	998	624	280	13367	107	4918	137	107	1
6	1	196	25	998	1062	158	3437	90	5068	120	90	1
7	1	196	25	1721	1061	161	5308	84	5559	114	84	1
8	1	196	25	1339	1414	177	176168	81	5615	111	81	1
9	1	196	25	1808	1524	190	229804	73	5454	103	73	1
10	1	196	25	996	1037	209	64637	77	5222	107	77	1

Slika 17. Tablica činjenica `fact_flight`

Slika 18 prikazuje konačnu strukturu dimenzije dim_route s generiranim surogat-ključem route_tk i jasno razdvojenim opisnim poljima. Ova dimenzija standardizira par polazište–odredište i pripadajuće geografske atribute: grad, državu i naziv aerodroma za polazište i odredište te udaljenost rute. Svaki je zapis jedinstven, prirodni ključ čini uređeni par (origin, destination) a surogatni ključ omogućuje stabilno povezivanje s tablicom činjenica u zvjezdastoj shemi.

	route_tk	origin	destination	departure_city	departure_country	departure_airport_name	destination_city	destination_country	destination_airport_name	distance
▶	1	EWR	ALB	Newark	United States	Newark Liberty International Airport	Albany	United States	Albany International Airport	143
	2	EWR	ANC	Newark	United States	Newark Liberty International Airport	Anchorage	United States	Ted Stevens Anchorage International Airport	3370
	3	EWR	ATL	Newark	United States	Newark Liberty International Airport	Atlanta	United States	Hartsfield Jackson Atlanta International Airport	746
	4	EWR	AUS	Newark	United States	Newark Liberty International Airport	Austin	United States	Austin Bergstrom International Airport	1504
	5	EWR	AVL	Newark	United States	Newark Liberty International Airport	Asheville	United States	Asheville Regional Airport	583
	6	EWR	BDL	Newark	United States	Newark Liberty International Airport	Hartford	United States	Bradley International Airport	116
	7	EWR	BNA	Newark	United States	Newark Liberty International Airport	Nashville	United States	Nashville International Airport	748
	8	EWR	BOS	Newark	United States	Newark Liberty International Airport	Boston	United States	Logan International Airport	200
	9	EWR	BQN	Newark	United States	Newark Liberty International Airport	Aguadilla	Puerto Rico	Rafael Hernández International Airport	1585
	10	EWR	BTW	Newark	United States	Newark Liberty International Airport	South Burlington	United States	Burlington International Airport	266

Slika 18. Dimenzija dim_route

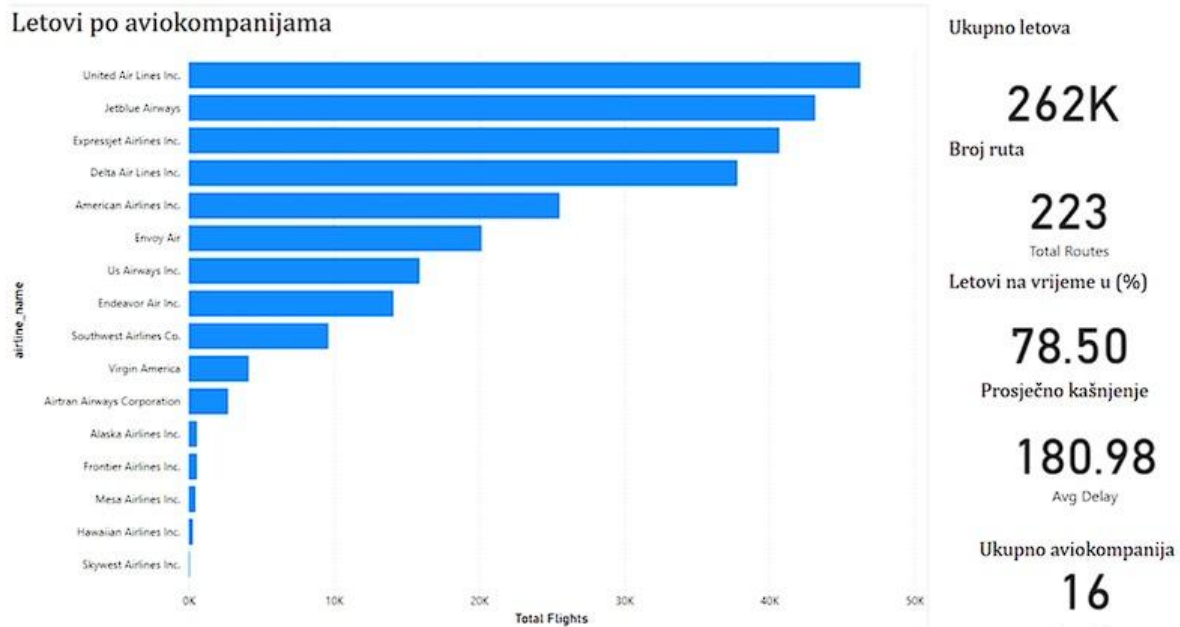
6. Vizualizacija i poslovna analiza

Nakon izgradnje skladišta podataka slijedi faza analize i vizualizacije u alatu Power BI. Cilj je sirove zapise o letovima pretvoriti u pregledne, interaktivne prikaze koji pomažu u brzom uočavanju obrazaca i donošenju odluka. Primijenjene su standardne OLAP operacije (Slice, Dice, Roll-up, Drill-down i Pivot), što omogućuje promatranje podataka iz više perspektiva od agregiranog pregleda do detalja po prijevozniku, ruti i vremenskom razdoblju.

6.1. Slice analiza

Prvi dashboard nudi pregled prometa njujorškog zračnog prostora u odabranom razdoblju te u zaglavlju ističe ključne pokazatelje: ukupno oko 262 tisuće letova, 223 rute, udio letova na vrijeme oko 78,5 posto, prosječno kašnjenje te broj avioprijevoznika (16). Ti indikatori daju brzi pregled sustava i služe kao polazište za daljnju analizu. U glavnom dijelu prikazan je vodoravni stupčasti graf s raspodjelom letova po aviokompanijama. Prikaz podržava slice filtriranje po prijevozniku, pa se isti raspored i KPI pokazatelji trenutno prilagođavaju odabiru.

Slika 19 prikazuje broj letova po avioprijevozniku i ključne KPI-je. Ovaj pregled otkriva strukturu tržišta (tko nosi najveći volumen) i upućuje gdje očekivati više kašnjenja te veći utjecaj na ukupne metrike. Relevantno je za BI nadzor i za pripremu značajke “prijevoznik” u predikcijskom modelu.



Slika 19. Cjeloukupan pregled aktivnosti aviokompanija

Slika 20 prikazuje KPI kartu s ukupnim brojem letova (~262 tisuće). Ovaj podatak potvrđuje veličinu i reprezentativnost analiziranog skupa te služi kao polazište za procjenu gustoće prometa i učestalosti kašnjenja, što je ključno za izgradnju pouzdanog prediktivnog modela.

Ukupno letova

262K

Slika 20. Ukupan broj letova svih aviokompanija

Slika 21 prikazuje KPI s ukupno 223 rute, čime se vidi širina i raznolikost mreže. To je bitno za razumijevanje geografske pokrivenosti i razlika u kašnjenjima među relacijama te opravdava uvođenje dimenzije “ruta” i značajki povezanih s udaljenošću i parom polazište–odredište.

Broj ruta

223

Total Routes

Slika 21. Ukupan broj ruta svih aviokompanija

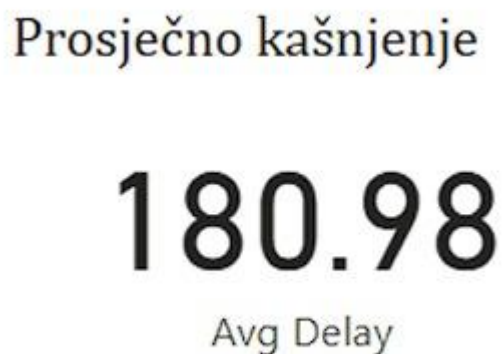
Slika 22 prikazuje KPI udio letova na vrijeme (~78,5 %), koji daje brzu sliku točnosti reda letenja i početnu razinu performansi sustava. Ovaj uvid služi kao referenca za usporedbu s rezultatima modela i procjenu poslovne vrijednosti eventualnog poboljšanja predikcija.

Letovi na vrijeme u (%)

78.50

Slika 22. Prosječno točnost letova

Slika 23 prikazuje prosječno kašnjenje od 180,98 minuta. Ovaj uvid pomaže prepoznati opseg problema kašnjenja u cjelini i daje početnu referencu za kasnije uspoređivanje stvarnih i predviđenih vrijednosti u prediktivnom modelu.



Slika 23. Sveukupno prosječno kašnjenje aviokompanija

6.2. Dice analiza

Drugi dashboard demonstrira kako kombiniranje više filtera preko više dimenzija stvara ciljani podskup podataka za usporedbe i brzo testiranje hipoteza. **Slice** primjenjuje jedan filter nad jednom dimenzijom i odmah mijenja broj letova, točnost i prosječno kašnjenje, **dice** istodobno uvjetuje, primjerice, prijevoznika, rutu i dio dana, čime fokusira analizu na točno definiran operativni scenarij.

Slika 24 prikazuje dashboard s kontrolama filtera i povezanim vizualima koji se dinamički ažuriraju, što čini učinke slice i dice operacija neposredno vidljivima.

OLAP operacije SLICE i DICE

Mjesec
month

1 12



Dan u tjednu
day_of_week

☒ Friday
☐ Monday
☐ Saturday
☐ Sunday
☐ Thursday
☐ Tuesday
☐ Wednesday

Aviokompanije
airline_name

☐ Airtran Airways Corporation
☐ Alaska Airlines Inc.
☐ American Airlines Inc.
☒ Delta Air Lines Inc.
☐ Endeavor Air Inc.
☐ Envoy Air
☐ Expressjet Airlines Inc.
☐ Frontier Airlines Inc.
☐ Hawaiian Airlines Inc.
☐ Jetblue Airways
☐ Mesa Airlines Inc.
☐ Skywest Airlines Inc.
☐ Southwest Airlines Co.
☐ United Air Lines Inc.
☐ Us Airways Inc.
☐ Virgin America

Ukupno letova

5525

Broj ruta

46

Total Routes

Letovi na vrijeme u (%)

81.94

Prosječno kašnjenje

76.57

Avg Delay

Ukupno aviokompanija

1

Slika 24. Vizualizacija SLICE i DICE

6.3. Roll-up i Drill-down analiza

Ovaj dashboard koristi hijerarhiju vremenske dimenzije s kretanjem po razinama. Na početnoj razini prikazuju se sažeti pokazatelji, primjerice približno 262 tisuće letova i prosječno kašnjenje oko 181 minute, što nudi “ptičju perspektivu”. Drill-downom do mjesečne razine uočavaju se sezonski obrasci viša kašnjenja u ljetnim mjesecima i stabilnije razdoblje od listopada do prosinca, dok je broj letova po mjesecu ujednačen, oko 21–23 tisuće. Dodatnim spuštanjem na dnevnu razinu pojavljuju se varijacije iz dana u dan i “kritični” datumi s pojačanim kašnjenjima. Roll-up natrag prema gore automatski re-agregira mjere prema DAX (Data Analysis Expressions) formulama, što omogućuje konzistentnu usporedbu između mjeseci i Godina [6].

Slika 25 prikazuje agregirane KPI pokazatelje i grafove na razini godine, namijenjene brzom orijentacijskom uvidu.

year	Total Flights	Avg Delay
2013	261877	180.98
Total	261877	180.98

Slika 25. Godišnji pregled (početna razina)

Slika 26 prikazuje raspodjelu po mjesecima s vidljivim sezonskim razlikama u kašnjenjima i stabilnim volumenom letova po mjesecu.

month	Total Flights	Avg Delay
1	21055	97.43
2	18926	96.26
3	22294	106.77
4	22055	111.35
5	22468	105.72
6	21599	126.27
7	22629	128.28
8	23118	103.03
9	21589	101.89
10	22924	88.04
11	21573	82.48
12	21647	109.17
Total	261877	180.98

Slika 26. Drill-down do mjesečne razine

Slika 27 prikazuje detaljan dnevni prikaz s oscilacijama po danima i naglašenim datumima povećanih kašnjenja.

day	Total Flights	Avg Delay
1	8635	87.81
2	8431	92.92
3	8891	78.12
4	8737	64.81
5	8484	83.28
6	8549	64.55
7	8546	89.14
8	8311	103.45
9	8192	82.40
10	8420	100.37
11	8788	89.14
12	8430	90.51
13	8723	83.53
14	8619	73.90
15	8903	66.64
16	8608	75.32
17	8762	86.70
18	8893	92.13
19	8656	87.88
Total	261877	180.98

Slika 27. Drill-down do dnevne razine

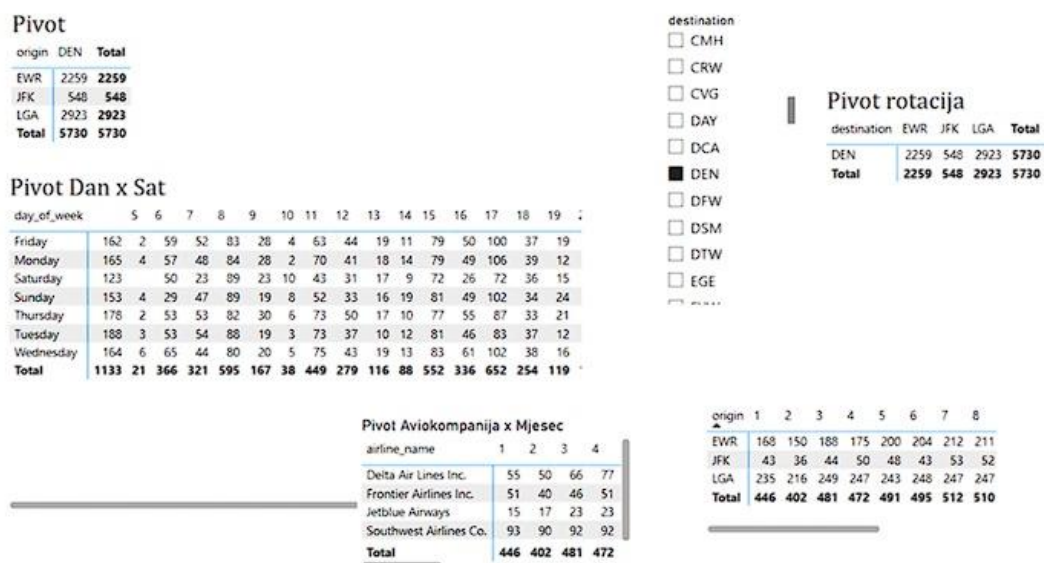
6.4. Pivot analiza

Ovaj dashboard prikazuje pivot analizu letova iz NYC područja filtriranih na odredište Denver (DEN). Rotiranjem dimenzija polazišnog aerodroma, sata polijetanja ili dana u tjednu dobiva se pregled broja letova i/ili prosječnog kašnjenja po odabranom presjeku. U primjeru NYC - DEN, LaGuardia (LGA) prednjači s 2 923 od ukupno 5 730 letova, najveći broj polazaka je između 17 i 18 sati, a najaktivniji su dani utorak i četvrtak. Ovakav pivot omogućuje brzo prepakiranje pogleda

bez dodatnog modeliranja i služi operativnoj optimizaciji rasporeda, kapaciteta gateova i posada u vršnim terminima.

Slika 28 prikazuje pivot tablicu koji mijenja redove i stupce ovisno o odabranoj dimenziji te odmah izračunava broj letova i prosječno kašnjenje po novonastalom presjeku.

Broj letova koji ide u DEN(Denver) iz svakog origin-a po mjesecima



Slika 28. PIVOT analiza

7. Razvoj predikcijskog modela

Model rješava binarnu klasifikaciju kašnjenja, pri čemu se let smatra „kasnim“ ako dolazi najmanje 15 minuta iza reda letenja. Učenje se provodi nad značajkama dostupnima prije polijetanja: mjesecom i danom u tjednu, planiranim vremenom polijetanja te iz njega izvedenim satom, identitetom prijevoznika, polazišnim i odredišnim aerodromom te udaljenošću rute, uz izvedene oznake vikenda i kategorije udaljenosti. Skup je podijeljen u omjeru 80:20 sa stratifikacijom ciljne varijable, kategorijski atributi kodirani su tehnikom LabelEncoder, a učinkovitost se primarno prati F1 mjerom zbog ravnoteže preciznosti i odziva. Tijekom treniranja promatra se kretanje F1 u funkciji veličine uzorka kao krivulja učenja te, radi konteksta, $1-F1$ kao aproksimacija krivulje gubitka. Operativni prag odluke od 0,50 po potrebi se prilagođava ako je cilj povećati recall. Završna evaluacija prikazuje se matricom zabune i tabličnim pregledom točnosti, preciznosti, odziva i F1, dok graf važnosti značajki daje uvid u relativni doprinos pojedinih varijabli. Detalji odabranog klasifikacijskog algoritma iznose se u sljedećem poglavlju.

7.1. Priprema i odabir podataka

Odabrane su značajke za koje se procjenjuje da najviše utječu na rizik kašnjenja (dolazak ≥ 15 min): vremenska obilježja poput sata planiranog polijetanja, mjeseca i dana u tjednu, geografsko-operativni kontekst kroz udaljenost rute te polazišni i odredišni aerodrom, kao i prijevoznik. Atribut „razlog kašnjenja“ zadržan je za deskriptivnu analizu i izostavljen iz prediktivnog skupa kada se ne može pouzdano poznavati ex-ante [12]. Podaci su očišćeni i standardizirani, uključujući parsiranje vremena u sat, kategorizaciju udaljenosti i označavanje vikenda, te su kodirani i pohranjeni u formatu pogodnom za treniranje i evaluaciju.

7.2. Odabir algoritma

S obzirom na to da je riječ o binarnoj klasifikaciji nad tabličnim podacima s mješavinom vremenskih, geografskih i operativnih atributa, odabran je ansambl stabala odluke Random Forest [9]. Takav pristup dobro podnosi nelinearne odnose i interakcije, ne zahtijeva skaliranje ulaza, prirodno radi s kategoriziranim varijablama kodiranim cijelim brojevima te pruža pregled važnosti

značajki koji olakšava interpretaciju rezultata i povezivanje s poslovnim uvidima [2]. Odabir Random Foresta predstavlja stabilnu polaznu točku kada su važnije robusnost i konzistentnost od osjetljivog ručnog podešavanja parametara.

7.3. Hiperparametri

Hiperparametri Random Forest modela određuju njegovu složenost, kapacitet učenja i robusnost te su presudni za uravnotežene performanse. U ovom radu podešavani su broj stabala (`n_estimators`), maksimalna dubina (`max_depth`), minimalni broj uzoraka za cijepanje čvora i formiranje lista (`min_samples_split`, `min_samples_leaf`), broj značajki dostupnih pri svakom cijepanju (`max_features`), način uzorkovanja (`bootstrap`), kriterij kvalitete cijepanja (`criterion`), tretman neuravnoteženih klasa (`class_weight`), stupanj paralelizacije (`n_jobs`) i slučajno sjeme (`random_state`). Broj stabala primarno utječe na stabilnost ansambla, maksimalna dubina kontrolira složenost i smanjuje prenaučенost, a minimalni brojevi uzoraka po razini i listu služe kao regularizatori. Ograničavanje `max_features` smanjuje korelaciju među stablima i povećava robusnost, `bootstrap` omogućuje internu procjenu pogreške, dok je `class_weight="balanced_subsample"` korišten kako bi se povisila osjetljivost na rjeđu, pozitivnu klasu (kašnjenja) [5].

Odabir vrijednosti vođen je kombinacijom iskustvenih preporuka i eksperimentalnog praćenja metrika [11]. Početno je korišteno 200 stabala uz umjereno ograničenje dubine (npr. `max_depth=10`); potom je broj stabala povećan na 300 radi stabilnosti, a ograničenje dubine postupno je relaksirano prema `None` kako bi model uhvatio kompleksnije obrasce bez vidljive prenaučенosti. `min_samples_leaf` postavljen je na 2 kao dodatna kontrola varijance, a sve konfiguracije trenirane su uz fiksirani `random_state` i paralelizaciju preko svih jezgri.

Za evaluaciju i usporedbu kombinacija hiperparametara primarno se pratila F1-mjera, jer uravnotežuje preciznost i odziv u uvjetima blage neuravnoteženosti klasa. Paralelno su promatrane točnost, preciznost i odziv po klasama te globalne mjere ROC AUC i, zbog informativnosti na neuravnoteženim skupovima, PR AUC [10]. Budući da je operativna cijena propuštenog kašnjenja veća od cijene lažne uzbune, dodatno je korištena F2-mjera koja dvostruko više vrednuje odziv od preciznosti. Na temelju krivulje preciznost–odziv odabran je prag koji maksimizira F2, čime se

kontrolirano povećava osjetljivost kada je to poslovno poželjno. Iako prag nije hiperparametar modela, njegovo se podešavanje provodi tek nakon što je utvrđena stabilna konfiguracija hiperparametara. Stabilnost postavki provjeravana je krivuljama učenja (F1 u ovisnosti o veličini skupa i broju stabala) te usporedbom rezultata na skupu za učenje i test, čime je postignut model s visokom točnošću, dobrim odzivom pozitivne klase i mogućnošću prilagodbe praga u skladu s operativnim prioritetima.

7.4. Postupak treniranja i praćenje učenja

Treniranje se provodi na stratificiranoj podjeli 80:20 kako bi se očuvao omjer klasa [11]. Primarna metrika optimizacije je F1, uz paralelno praćenje preciznosti, odziva i točnosti. Dinamika učenja prati se krivuljom učenja, gdje se F1 promatra u ovisnosti o veličini skupa za učenje, te pomoćnom „krivuljom gubitka“ u kojoj se varira broj stabala uz fiksne ostale postavke radi uvida u konvergenciju modela [3]. Kada je to operativno opravdano, prag odluke se podešava kako bi se postigla veća osjetljivost na stvarna kašnjenja.

Uz navedene metrike, u ovom radu se posebno prati i F2, jer u zadatku procjene kašnjenja cijena propuštenog kašnjenja (lažno negativan slučaj) veća je od cijene lažne uzbune. Za razliku od F1 koja jednako važe preciznost i odziv, F2 daje dvostruku težinu odzivu, pa se prag odluke ne postavlja proizvoljno nego se bira s krivulje preciznost–odziv na točki koja maksimizira F2 [5].

7.5. Bazni model

U početnoj iteraciji skripta učitava skup `flights_main.csv`, parsira planirano vrijeme polijetanja u minute od ponoći i izvodi sat polijetanja, označava vikend, kategorizira udaljenost rute na kratku, srednju i dugu te binarizira cilj `delayed` prema DOT (U.S. Department of Transportation) pragu od 15 minuta zakašnjenja na dolasku [4]. Kategorijski atributi kodirani su `LabelEncoderom`, a zatim je izrađena stratificirana podjela 80:20 na skup za učenje i test. Kao početni klasifikator trenira se `RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)`. Nakon učenja generira se izvještaj klasifikacije, prikazuju se matrica zabune i važnosti značajki, a prate se i pomoćni EDA grafovi te funkcije za procjenu vjerojatnosti kašnjenja na novim letovima.

Slika 29 prikazuje dio skripte i izlaze koji potvrđuju ispravno parsiranje vremena, označavanje vikenda, kategorizaciju udaljenosti i formiranje ciljne varijable `delayed` prema pragu od 15 minuta. Također je vidljiva provjera dimenzija i osnovna statistika koja potvrđuje spremnost skupa za treniranje.

```
def parse_time(val):
    """HHMM ili 'HH:MM' -> minute od ponoći; nevaljano -> NaN."""
    try:
        if isinstance(val, str) and ":" in val:
            h, m = val.split(":")
            return int(h) * 60 + int(m)
        val = str(int(val)).zfill(4)
        return int(val[:2]) * 60 + int(val[2:])
    except Exception:
        return np.nan

df["sched_dep_time"] = df["sched_dep_time"].apply(parse_time)
df["sched_dep_hour"] = df["sched_dep_time"] // 60

def is_weekend_val(x):
    if isinstance(x, str):
        return 1 if x.lower() in {"sat", "saturday", "sun", "sunday"} else 0
    return 1 if x in [6, 7, "6", "7"] else 0

df["is_weekend"] = df["day_of_week"].apply(is_weekend_val).astype(int)

def distance_cat(dist):
    if pd.isna(dist): return "unknown"
    if dist < 500: return "short"
    elif dist < 1500: return "medium"
    else: return "long"

df["route_distance_cat"] = df["distance"].apply(distance_cat)

# >>> Target: kasni > 15 min (DOT)
df["delayed"] = (pd.to_numeric(df["arr_delay_time"], errors="coerce") > THRESHOLD_MINUTES).astype(int)
```

Slika 29. Priprema ulaza i binarni cilj za bazni model

Slika 30 prikazuje konstrukciju i treniranje početnog Random Forest modela na tren skupu te spremanje predikcija i vjerojatnosti za test skup. Vidljivo je evidentiranje hiperparametara, fiksiranje slučajnog sjemena i izdvajanje rezultata potrebnih za kasniju evaluaciju.

```

# Značajke i split

features = [
    "month", "day_of_week", "sched_dep_hour", "reason_dep_delay",
    "carrier", "origin", "destination", "distance", "is_weekend", "route_distance_cat"
]
X = df[features]
y = df["delayed"]

stratify = y if STRATIFY_SPLIT else None
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=stratify
)

# RandomForest klasifikacija

model = RandomForestClassifier(
    n_estimators=200, max_depth=10, random_state=RANDOM_STATE
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("\n--- Model Performance (Classification, target: >15 min) ---")
print(classification_report(y_test, y_pred, zero_division=0, target_names=["na_vrijeme", "kasnio"]))

ConfusionMatrixDisplay.from_estimator(
    model, X_test, y_test, display_labels=["Na vrijeme", "Kasnio"], values_format="d"
)
plt.title("Confusion matrix - RandomForest")
plt.tight_layout()
plt.show()

```

Slika 30. Treniranje klasifikatora

Na testnom skupu bazni model postiže ukupnu točnost 0,86. Za klasu „na vrijeme“ ostvaruje se preciznost 0,86, odziv 0,98 i F1 0,92, dok za klasu „kasnio“ preciznost iznosi 0,87, a odziv 0,49, uz F1 0,63 (macro-avg F1 približno 0,77). Matrica zabune pokazuje povećan broj propuštenih kašnjenja (FN), zbog čega je osjetljivost preniska za operativnu uporabu bez dodatnog podešavanja. **Zaključeno** je da bazni model treba **povećati** recall za klasu „kasnio“, primjerice finim podešavanjem hiperparametara, uvođenjem `class_weight='balanced'` i/ili pomakom praga odluke te sustavnim praćenjem metrika na validaciji.

Slika 31 prikazuje matricu zabune i izvještaj klasifikacije s metrikama točnosti, preciznosti, odziva i F1 po klasama. Vizualno je uočljivo da se najveće pogreške javljaju u propuštanju stvarnih kašnjenja, što usmjerava sljedeće iteracije prema povećanju odziva pozitivne klase. Ova početna verzija poslužila je kao referentna točka za poboljšanje modela, uključujući balansiranje podataka i podešavanje hiperparametara.

```

--- Model Performance (Classification, target: >15 min) ---
              precision    recall  f1-score   support

na_vrijeme    0.86        0.98        0.92    39942
kasnio        0.87        0.49        0.63    12434

accuracy              0.86    52376
macro avg    0.87        0.74        0.77    52376
weighted avg    0.86        0.86        0.85    52376

```

Slika 31. Evaluacija prve verzije modela

7.6. Finalni predikcijski model

Cilj poboljšane verzije bio je povećati osjetljivost na letove koji doista kasne uz što manji gubitak ukupne točnosti i preciznosti. Polazišni model, iako ukupno točan, propuštao je previše pozitivnih slučajeva, pa je operativna vrijednost za pravodobnu reakciju bila ograničena. U ovoj iteraciji kombinirano je uravnoteženje klasa tijekom učenja, umjereno povećanje kapaciteta modela kako bi se zahvatilo više obrazaca te promišljeno podešavanje praga odluke na temelju krivulje precision–recall. Tom kombinacijom postignut je znatno viši recall klase „kasnio“ uz zadržavanje vrlo dobre ukupne točnosti i razumne razine lažnih uzbuna.

7.6.1. Konfiguracija modela

Slika 32 prikazuje model koj najprije postavlja radno okruženje i globalne konstante: putanju do ulaznih datoteka, DOT prag od 15 minuta, omjer podjele skupa te random_state radi ponovljivosti eksperimenata. Time se osigurava kontrolirano i reproduktivno izvođenje svih sljedećih koraka.

```
# Konfiguracija
PATH = "flights_main.csv"
THRESHOLD_MINUTES = 15
TEST_SIZE = 0.20
RANDOM_STATE = 42
BEST_THRESHOLD = 0.5
```

Slika 32. Okruženje i globalne postavke

Slika 33 prikazuje učitavanje samo potrebnih stupaca za predviđanje i evaluaciju, čime se smanjuje potrošnja memorije i ubrzava izvođenje. Takvo selektivno čitanje skraćuje ETL tijekom učenja modela i olakšava ponovna pokretanja. Na slici je vidljiv odabir relevantnih varijabli za daljnju pripremu.

```
# Učitavanje podataka
usecols = ["month", "day_of_week", "sched_dep_time", "reason_dep_delay",
           "carrier", "origin", "destination", "distance", "arr_delay_time"]
df = pd.read_csv(PATH, usecols=usecols)
```

Slika 33. Učitavanje i selekcija stupaca

Slika 34 prikazuje funkciju za parsiranje vremena koja zapise u formatu HHMM ili HH:MM pretvara u minute od ponoći, a iz njih izvodi sat polijetanja (sched_dep_hour). Tako dobiven atribut pouzdano hvata dnevne obrasce prometa i kašnjenja.

```
# priprema značajki
def parse_time(v):
    try:
        if isinstance(v, str) and ":" in v:
            h, m = v.split(":"); return int(h)*60 + int(m)
        s = str(int(v)).zfill(4); return int(s[:2])*60 + int(s[2:])
    except Exception:
        return np.nan

df["sched_dep_time"] = df["sched_dep_time"].apply(parse_time)
df["sched_dep_hour"] = df["sched_dep_time"] // 60
```

Slika 34. Parsiranje vremena i izvođenje sata polijetanja

Slika 35 prikazuje funkciju is_weekend koja dan u tjednu pretvara u indikator vikenda. Na taj način model može razlikovati vikend od radnih dana i uočiti specifične operative uvjete povezane s vikendom.

```
# priprema značajki
def parse_time(v):
    try:
        if isinstance(v, str) and ":" in v:
            h, m = v.split(":"); return int(h)*60 + int(m)
        s = str(int(v)).zfill(4); return int(s[:2])*60 + int(s[2:])
    except Exception:
        return np.nan

df["sched_dep_time"] = df["sched_dep_time"].apply(parse_time)
df["sched_dep_hour"] = df["sched_dep_time"] // 60
```

Slika 35. Oznaka vikenda

Slika 36 prikazuje mapiranje udaljenosti rute u diskretne kategorije (kratka, srednja, duga). Ovakvo grupiranje pomaže modelu da prepozna nelinearne razlike među rutama bez potrebe za ručnim određivanjem pragova.

```
def distance_cat(dist):
    if pd.isna(dist): return "unknown"
    if dist < 500: return "short"
    elif dist < 1500: return "medium"
    else: return "long"

df["route_distance_cat"] = df["distance"].apply(distance_cat)

# target: >15 min
df["delayed"] = (pd.to_numeric(df["arr_delay_time"], errors="coerce") > THRESHOLD_MINUTES).astype(int)
```

Slika 36. Kategorizacija udaljenosti rute

Slika 37 prikazuje definiranje ciljne varijable delayed: vrijednost je 1 ako je kašnjenje na dolasku veće od 15 minuta, u skladu s DOT praksom. Time se jasno određuje pozitivna klasa.

```
# cilj: >15 min
df["delayed"] = (pd.to_numeric(df["arr_delay_time"], errors="coerce") > THRESHOLD_MINUTES).astype(int)
```

Slika 37. Definicija ciljne varijable

slika 38 Kategorijski atributi kodiraju se u cijele brojeve tehnikom LabelEncoder. Enkoderi se čuvaju u rječniku kako bi se u inferenciji nad novim letovima primijenilo potpuno isto mapiranje.

```
# Label encoding
cat_cols = ["reason_dep_delay", "carrier", "origin", "destination", "day_of_week", "route_distance_cat"]
le_dict = {}
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    le_dict[col] = le
```

Slika 38. Label encoding kategorija

Slika 39 prikazuje formiranje matrice značajki nakon odabira atributa te stratificiranu podjelu skupa podataka u omjeru 80:20. Ovim korakom završava priprema podataka i započinje faza treniranja.

```
# Značajke + split training/test
features = ["month", "day_of_week", "sched_dep_hour", "reason_dep_delay",
            "carrier", "origin", "destination", "distance", "is_weekend", "route_distance_cat"]
X = df[features]
y = df["delayed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)
```

Slika 39. Značajke i stratificirani split

Slika 40 prikazuje konačni klasifikator – šumu nasumičnih stabala s uravnoteženjem klasa na razini bootstrap uzorka. Veći broj stabala (npr. 300) doprinosi stabilnosti, dopuštena veća dubina hvata složenije interakcije, minimalni broj uzoraka po listu smanjuje prenaučenos, a `class_weight="balanced_subsample"` povećava osjetljivost na kašnjenja. Paralelizacija (`n_jobs=-1`) ubrzava učenje [8].

```
# Model
rf = RandomForestClassifier(
    n_estimators=300, max_depth=None, min_samples_leaf=2,
    class_weight="balanced_subsample", n_jobs=-1, random_state=RANDOM_STATE
)
rf.fit(X_train, y_train)
```

Slika 40. Definicija modela i hiperparametara

Slika 41 prikazuje baznu evaluaciju: nakon treniranja izračunavaju se vjerojatnosti i klasne predikcije pri standardnom pragu 0,50, ispisuju se metrike po klasama i crta matrica zabune. Time se dobiva polazna točka prije bilo kakvog podešavanja praga.

```
# baseline @ 0.5
y_proba = rf.predict_proba(X_test)[: , 1]
y_pred_05 = (y_proba >= 0.5).astype(int)
print("\n--- Report @0.5 ---")
print(classification_report(y_test, y_pred_05, zero_division=0, target_names=["na_vrijeme", "kasnio"]))
plot_cm_heatmap(y_test, y_pred_05, title="Confusion matrix - RF @0.5")
```

Slika 41. Report na pragu 0.5 i matrica zabune

U ovom zadatku operativno je važnije na vrijeme prepoznati stvarna kašnjenja nego izbjegnuti lažne uzbune, jer propušteno kašnjenje pokreće kaskadne troškove (disrupcije gateova i posada, zadržavanja putnika, domino-efekte na sljedeće rotacije). To znači da je **lažno negativan** slučaj skuplji od **lažno pozitivnog**, pa metrika koja jače naglašava **odziv** bolje odražava cilj. Zbog toga se uz F1 dodatno koristi **F2**, koja recallu daje dvostruku težinu u odnosu na preciznost te se **prag odluke** ne postavlja proizvoljno, nego se bira s **krivulje precision–recall** na točki koja **maksimizira F2**. Na taj način se granica odlučivanja svjesno pomiče prema većoj osjetljivosti, uz transparentno prihvaćanje da će se dio preciznosti izgubiti.

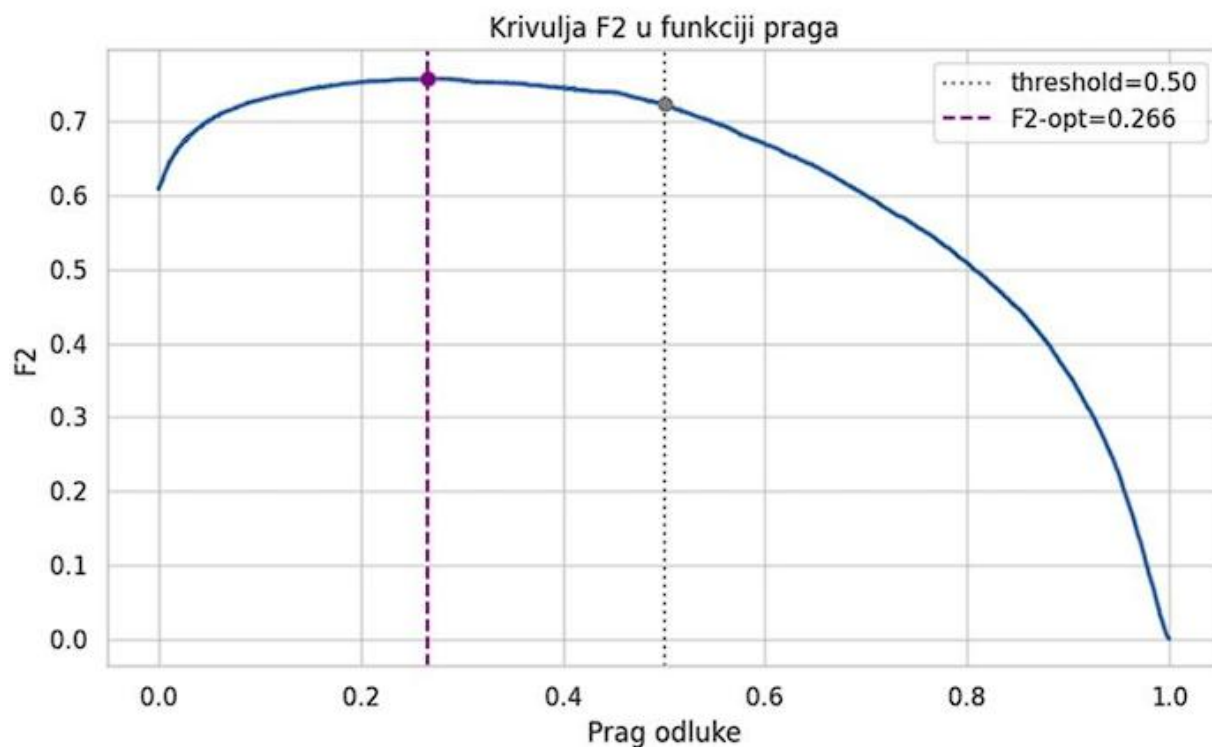
Slika 42 prikazuje odabir praga odluke prema F2-mjeri ($\beta = 2$). Na temelju krivulje preciznost–odziv pronađen je prag oko 0,245 na kojem recall klase „kasnio“ doseže oko 0,87, uz preciznost približno 0,50 i ukupnu točnost oko 0,77. Ipak, za redovnu primjenu zadržan je standardni prag 0,50 (recall $\approx 0,72$ i preciznost $\approx 0,74$) jer bolje uravnotežuje preciznost i osjetljivost. F2-optimalni prag ostaje koristan u scenarijima kada je važnije na vrijeme prepoznati što više kašnjenja, iako to znači veći broj lažnih uzbuna.

```
# tuning praga: F2
precision, recall, thresholds = precision_recall_curve(y_test, y_proba)
beta = 2.0
f2 = (1+beta*2) * (precision[:-1]*recall[:-1]) / (beta*2*precision[:-1] + recall[:-1] + 1e-12)
BEST_THRESHOLD = float(thresholds[np.nanargmax(f2)])
y_pred_best = (y_proba >= BEST_THRESHOLD).astype(int)

print(f"\nOdabrani prag (F2): {BEST_THRESHOLD:.3f}")
print("\n--- Report @F2-opt ---")
print(classification_report(y_test, y_pred_best, zero_division=0, target_names=["na_vrijeme", "kasnio"]))
plot_cm_heatmap(y_test, y_pred_best, title="Confusion matrix - RF @F2-opt")
```

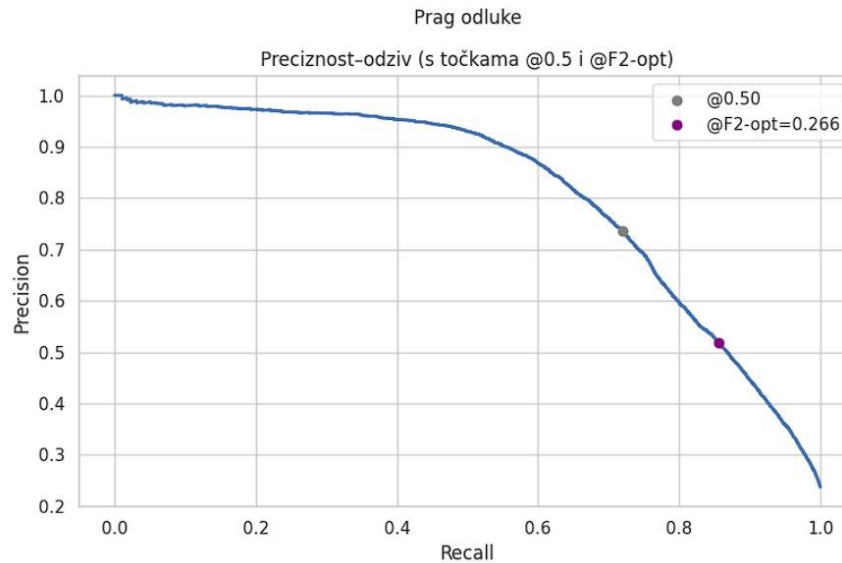
Slika 42. Odabrani F2 prag i matrica

Slika 43 krivulje prikazuje kako se F2 ($\beta=2$) mijenja s pragom odluke. Okomita isprekidana linija na 0,50 označuje standardni prag, a ljubičasta linija vrijednost oko 0,266, gdje F2 doseže lokalni maksimum (plato). To je poslužilo kao osnova za izbor F2-optimalne točke rada, uz napomenu da pragovi u uskom rasponu oko 0,266 postižu vrlo slične F2 vrijednosti.



Slika 43. F2 u funkciji praga odluke

Slika 44 prikazuje odnos preciznosti i odziva za sve pragove. Označene su točke @0,50 i @F2-opt $\approx$ 0,266. Pomak s 0,50 na F2-opt povećava recall uz očekivani pad preciznosti, što ilustrira kompromis pri odabiru praga.



Slika 44. Krivulja preciznost–odziv s označenim pragovima

Slika 45 prikazuje važnost značajki mjerenu metodom Mean Decrease in Impurity (MDI), koja je standardni postupak za Random Forest. Svako stablo pri svakom cijepanju smanjuje Gini nečistoću, a ta se smanjenja zbrajaju za svaku značajku i potom normaliziraju kroz cijelu šumu. Dobivena vrijednost pokazuje relativni doprinos pojedine značajke ukupnom smanjenju nečistoće (izražen u postotku ili na skali 0–1), uzimajući u obzir i učestalost korištenja značajke i veličinu poboljšanja koje donosi pri cijepanju čvorova.

```
# Feature importance
importances = pd.Series(rf.feature_importances_, index=features).sort_values(ascending=False)
imp_df = importances.reset_index(); imp_df.columns = ["feature", "importance"]
barplot_gradient(
    imp_df, x="importance", y="feature", hue="feature", cmap="magma",
    title="Feature Importance (Random Forest)", xlabel="Važnost", ylabel="Značajka"
)
```

Slika 45. Važnost značajki

Slika 46 prikazuje produkcijski sloj koji prima opis budućeg leta, provodi iste transformacije kao u fazi učenja i vraća vjerojatnost kašnjenja te binarnu odluku prema odabranom pragu. Funkcija za kodiranje parsira planirano vrijeme i izvodi sat polijetanja, računa indikator vikenda, kategorizira udaljenost te primjenjuje label-encoding istim enkoderima, uz siguran postupak za nepoznate vrijednosti. Funkcija za predikciju potom izračuna vjerojatnost iz modela i vraća par (postotak, klasa) prema F2-optimalnom ili operativno odabranom pragu. Kratki demo s tri leta pokazuje kako se procjene razlikuju ovisno o ruti, dobu dana i operativnom kontekstu.

```
# Helperi za predikciju na novim letovima
def encode_row(row: pd.DataFrame) -> pd.DataFrame:
    row = row.copy()
    if "sched_dep_time" in row:
        row["sched_dep_time"] = row["sched_dep_time"].apply(parse_time)
        row["sched_dep_hour"] = row["sched_dep_time"] // 60
    if "day_of_week" in row:
        row["is_weekend"] = row["day_of_week"].apply(
            lambda x: 1 if (str(x).lower() in {"sat", "saturday", "sun", "sunday"}) or x in [6, 7, "6", "7"]) else 0
    )
    if "distance" in row:
        def _dc(v):
            if pd.isna(v): return "unknown"
            if v < 500: return "short"
            elif v < 1500: return "medium"
            else: return "long"
        row["route_distance_cat"] = row["distance"].apply(_dc)
    for col in ["reason_dep_delay", "carrier", "origin", "destination", "day_of_week", "route_distance_cat"]:
        if col in row:
            le = le_dict[col]
            known = set(le.classes_)
            vals = row[col].astype(str).apply(lambda x: x if x in known else le.classes_[0])
            row[col] = le.transform(vals)
    return row

def predict_delay_percent(new_data: dict, threshold: float = None):
    """Vrati (postotak, klasa) prema F2 pragu (ili custom thr)."""
    row = pd.DataFrame([new_data]); row_enc = encode_row(row)
    proba = rf.predict_proba(row_enc[features])[0][1]
    thr = BEST_THRESHOLD if threshold is None else threshold
    return round(proba * 100, 2), int(proba >= thr)

# Demo letovi
new_flights = [
    {"month": 5, "day_of_week": "Monday", "sched_dep_time": "08:30", "reason_dep_delay": "Air Traffic Congestion", "carrier": "UA", "origin": "JFK", "destination": "MIA", "distance": 1089},
    {"month": 12, "day_of_week": "Sunday", "sched_dep_time": "18:40", "reason_dep_delay": "Early Departure", "carrier": "DL", "origin": "LGA", "destination": "MYR", "distance": 50},
    {"month": 12, "day_of_week": "Sunday", "sched_dep_time": "18:40", "reason_dep_delay": "On Time", "carrier": "DL", "origin": "LGA", "destination": "MYR", "distance": 800},
]
```

Slika 46. Helperi za inference i demo letova

8. Rezultati i evaluacija modela

U ovom poglavlju prikazuje se cjelovita evaluacija finalnog modela za predikciju kašnjenja leta. Procjena je rađena na nezavisnom testnom skupu (20% uz stratifikaciju). Uz osnovnu procjenu na zadanom pragu 0,50, prikazana je i varijanta s pragom optimiranim na F2-mjeru (naglasak na recall pozitivne klase „kasnio”). Na kraju se donose opći pokazatelji diskriminativnosti modela (ROC AUC, PR AUC), interpretacija važnosti značajki te nekoliko ilustrativnih primjera inferencije.

8.1. Klasifikacijske metrike

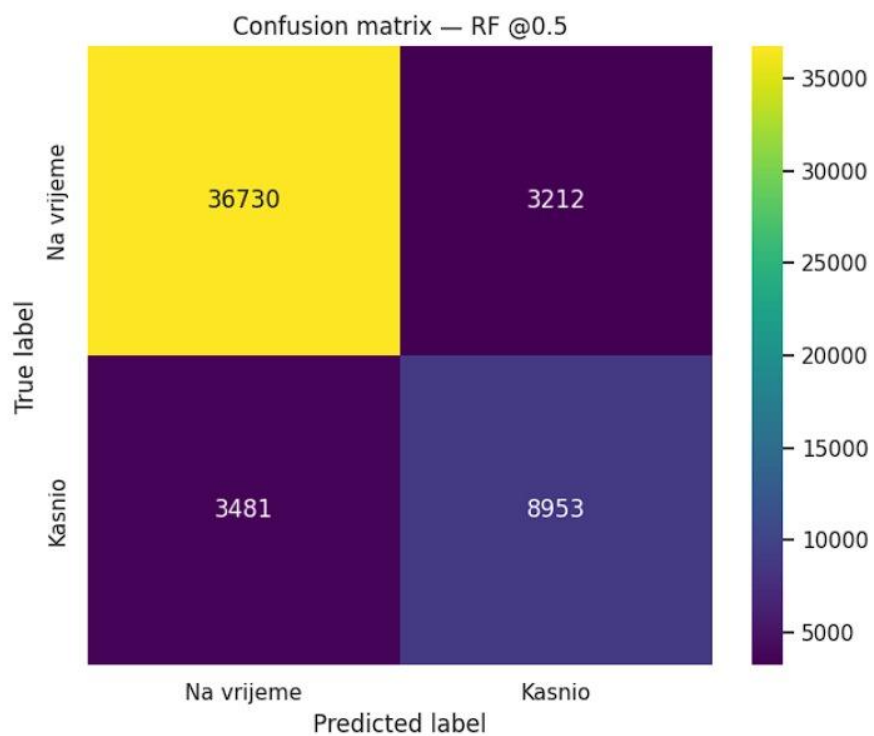
Slika 47 prikazuje završne rezultate modela pri pragu 0,50, gdje ukupna točnost iznosi 0,87. Za klasu „na vrijeme“ izmjerene su vrijednosti precision 0,91, recall 0,92 i F1 0,92, dok su za klasu „kasnio“ precision 0,74, recall 0,72 i F1 0,73 (macro F1 $\approx$ 0,82). Ovakav prag osigurava uravnoteženo prepoznavanje oba razreda i dobru ukupnu učinkovitost modela.

	precision	recall	f1-score	support
na_vrijeme	0.91	0.92	0.92	39942
kasnio	0.74	0.72	0.73	12434
accuracy			0.87	52376
macro avg	0.82	0.82	0.82	52376
weighted avg	0.87	0.87	0.87	52376

Slika 47. Klasifikacijski izvještaj (rezultati modela)

8.2. Matrica zabune

Slika 48 prikazuje matricu zabune na testnom uzorku od 52 376 letova: ispravno je prepoznato 36 730 letova „na vrijeme” i 8 953 „kasnih”, uz 3 212 lažno pozitivnih i 3 481 lažno negativnih. Prikaz potvrđuje da je broj propuštenih kašnjenja (FN) pri ovom pragu relativno nizak.



Slika 48. Matrica zabune

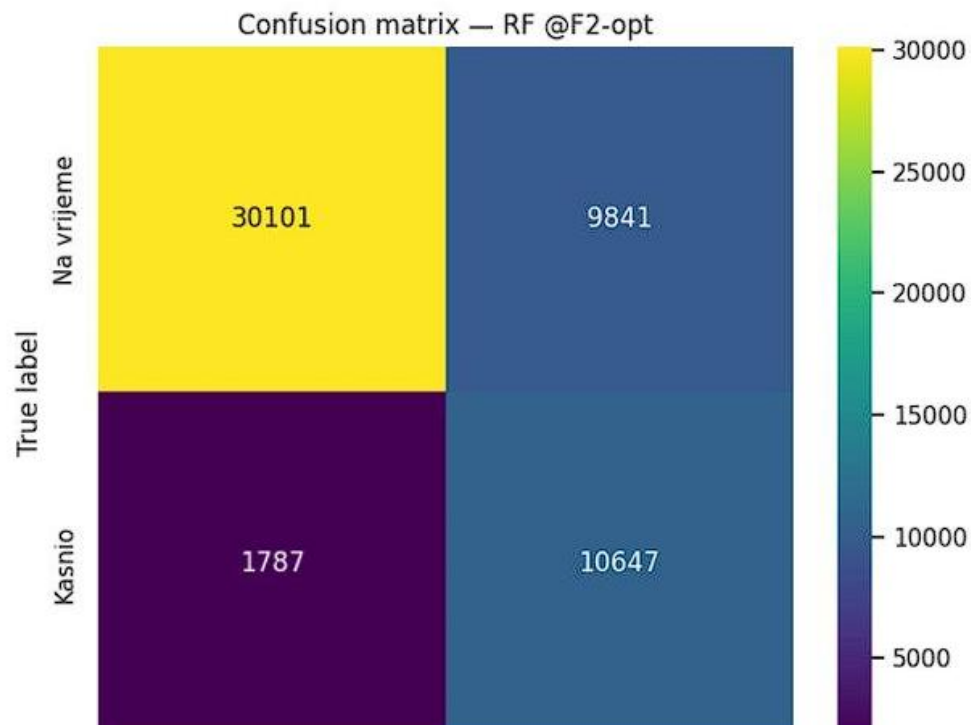
8.3. Optimizacija praga na F2

Slika 49 prikazuje rezultat optimizacije praga preko krivulje preciznost–odziv: odabran je prag oko 0,266. Na toj točki recall za klasu „kasnio“ raste na 0,86 (precision 0,52, F1 0,65), dok za klasu „na vrijeme“ recall iznosi 0,75 (precision 0,94, F1 0,84). Ukupna točnost tada je 0,78 [7]. Ovakva postavka smanjuje broj propuštenih kašnjenja (FN s 3 481 na 1 787), ali povećava broj lažnih uzbuna (FP s 3 212 na 9 841).

Odabrani prag (F2): 0.266

--- Report @F2-opt ---

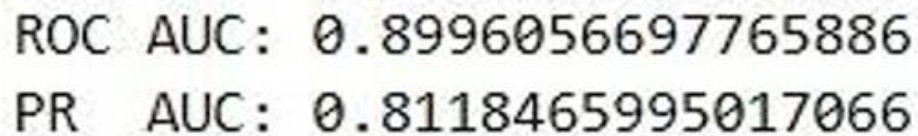
	precision	recall	f1-score	support
na_vrijeme	0.94	0.75	0.84	39942
kasnio	0.52	0.86	0.65	12434
accuracy			0.78	52376
macro avg	0.73	0.80	0.74	52376
weighted avg	0.84	0.78	0.79	52376



Slika 49. Klasifikacijski izvještaj za prag F2 i matrica zabune

8.4. Globalne mjere diskriminativnosti

Slika 50 prikazuje globalne mjere diskriminativnosti modela: površina ispod ROC krivulje iznosi oko 0,90, a prosječna preciznost (PR AUC) oko 0,81. Kako je skup blago neuravnotežen, PR AUC je informativniji pokazatelj te potvrđuje dobru sposobnost modela da pri višim vjerojatnostima „sabere” istinske pozitivne primjere (kasnio).

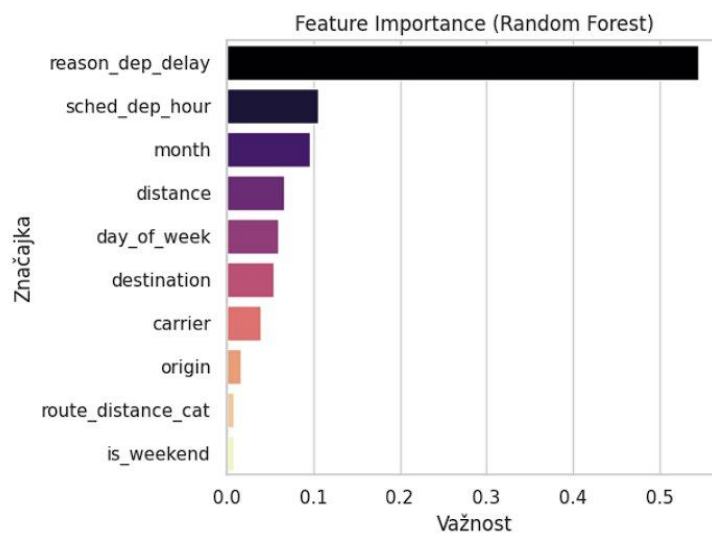


ROC AUC: 0.8996056697765886
PR AUC: 0.8118465995017066

Slika 50. ROC AUC i PR AUC

8.5. Važnost značajki

Slika 51 prikazuje rangiranje značajki prema njihovom doprinosu odluci modela. Najveći utjecaj imaju sat planiranog polijetanja (sched_dep_hour), mjesec, prijevoznik i udaljenost rute, dok ostale značajke daju manji, ali stabilan doprinos. Ako je neka značajka post-event naravi i nije poznata prije polijetanja (npr. stvarni razlog odgode), u produkciji je treba izostaviti ili zamijeniti varijablama dostupnima ex-ante.



Slika 51. Važnost značajki

8.6. Predikcija na novim letovima

Slika 52 prikazuje rezultate na tri simulirana leta: procijenjene vjerojatnosti kašnjenja iznose približno 94,5 %, 18,3 % i 31,9 %. Pri F2-optimiziranom pragu odluke su 1, 0 i 1 – prvi let ima vrlo visoku, gotovo sigurnu vjerojatnost kašnjenja, drugi najnižu i očekuje se na vrijeme, a treći nosi umjeren rizik. Takav izlaz pomaže procijeniti operativni rizik po ruti i dobu dana te planirati ciljane intervencije.

```
Flight 1: P(delay>15) = 94.51% | class@F2 = 1
Flight 2: P(delay>15) = 18.32% | class@F2 = 0
Flight 3: P(delay>15) = 31.93% | class@F2 = 1
```

Slika 52. Rezultati predikcije triju letova

9. Zaključak

Ovim radom pokazano je kako se podaci o letovima mogu pretvoriti u koristan alat za planiranje i predviđanje. Najprije su prikupljeni i očišćeni svi zapisi, a zatim je izrađeno skladište podataka koje čuva sve informacije na jednom mjestu, uklanja greške i osigurava da su svi podaci međusobno usklađeni. Na temelju tog skladišta izrađen je dimenzijski model i proveden ETL proces, čime su podaci pripremljeni za analizu i prikaz u interaktivnim BI vizualizacijama. Ovi prikazi omogućuju brzo uočavanje uzoraka u prometu, kašnjenjima i točnosti letova.

Skladište podataka pokazalo se kao ključni dio rješenja jer daje jedinstvenu i provjerenu osnovu za sve daljnje analize i predikcije. Ono osigurava da se u predikcijski model unose samo podaci koji su stvarno poznati prije polijetanja, čime se sprječava pogrešna procjena i postiže pouzdanost rezultata.

Na toj osnovi izrađen je predikcijski model koji procjenjuje vjerojatnost da će let kasniti 15 minuta ili više. Njegova je svrha pravodobno upozoriti na moguće kašnjenje kako bi se na vrijeme prilagodili gateovi, posade i obavijestili putnici. Model temeljen na Random Forest algoritmu pokazao je dobru točnost i uravnotežen odnos preciznosti i odziva, a dodatna optimizacija praga F2 omogućila je još veću osjetljivost na stvarna kašnjenja kada je to potrebno. Na tri probna leta model je procijenio vjerojatnost kašnjenja na 94,5 %, 18,3 % i 31,9 %, pri čemu su odluke bile 1, 0 i 1, što znači da se prvi let gotovo sigurno očekuje s kašnjenjem, drugi na vrijeme, a treći nosi umjeren rizik.

Cjelokupno rješenje od skladišta podataka, preko BI vizualizacija do prediktivnog modela pokazuje snagu povezivanja poslovne inteligencije i strojnog učenja. Skladište daje sigurnu i čistu osnovu, vizualizacije pomažu u brzom uočavanju obrazaca, a model omogućuje proaktivno upravljanje redom letenja i bolju organizaciju zračnog prometa.

Literatura

- [1] Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.
- [2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- [3] Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.
- [4] U.S. DOT / BTS. Airline On-Time Statistics and Delay Causes .
- [5] scikit-learn. Model evaluation: precision, recall, F-score (F_1 , F_β), precision-recall curve.
- [6] Microsoft Learn. Data Analysis Expressions (DAX), overview & reference (Power BI).
- [7] Apache Spark. Spark SQL, DataFrames and Datasets Guide.
- [8] MySQL. FOREIGN KEY Constraints.
- [9] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
- [10] Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
- [11] Kuhn, M., & Johnson, K. (2013). Applied Predictive Modeling. Springer.
- [12] Kuhn, M., & Johnson, K. (2019). Feature Engineering and Selection: A Practical Approach for Predictive Models. CRC Press.

¹Potpuni kod i prateći materijali dostupni su na GitHub repozitoriju:

https://github.com/Gabrieln99/Zavrzni_rad

Popis slika

Slika 1 Početna analiza dataseta	4
Slika 2. Dimenzije skupa podataka	5
Slika 3. Nedostajućih vrijednosti po stupcu	6
Slika 4. Broj jedinstvenih vrijednosti po stupcu	7
Slika 5. Skripta za predprocesiranje skupa podataka i podjelu 80:20	9
Slika 6. Rezultat predprocesiranja i podjele (80:20)	10
Slika 7. ER dijagram relacijskog modela	12
Slika 8. Relacijski model baze podataka	13
Slika 9. Definiranje sheme baze podataka	15
Slika 10. Primjer punjenja tablica (airline, route)	16
Slika 11. Dimenzijski model podataka	18
Slika 12. extract_csv.py	20
Slika 13. extract_csv.py	21
Slika 14. Normalizacija MySQL i CSV izvora, standardizacija i union bez duplikata	23
Slika 15. Generiranje surogat ključa i formiranje izlazne sheme dimenzije	24
Slika 16. Primjer funkcije za učitavanje podataka	25
Slika 17. Tablica činjenica fact_flight	26
Slika 18. Dimenzija dim_route	27
Slika 19. Cjelokupan pregled aktivnosti aviokompanija	29
Slika 20. Ukupan broj letova svih aviokompanija	29
Slika 21. Ukupan broj ruta svih aviokompanija	30
Slika 22. Prosječno točnost letova	30
Slika 23. Sveukupno prosječno kašnjenje aviokompanija	31
Slika 24. Vizualizacija SLICE i DICE	32
Slika 25. Godišnji pregled (početna razina)	33
Slika 26. Drill-down do mjesečne razine	33
Slika 27. Drill-down do dnevne razine	34
Slika 28. PIVOT analiza	35
Slika 29. Priprema ulaza i binarni cilj za bazni model	39
Slika 30. Treniranje klasifikatora	40
Slika 31. Evaluacija prve verzije modela	41
Slika 32. Okruženje i globalne postavke	42
Slika 33. Učitavanje i selekcija stupaca	42
Slika 34. Parsiranje vremena i izvođenje sata polijetanja	43
Slika 35. Oznaka vikenda	43
Slika 36. Kategorizacija udaljenosti rute	44

Slika 37. Definicija ciljne varijable	44
Slika 38. Label encoding kategorija.....	44
Slika 39. Značajke i stratificirani split.....	45
Slika 40. Definicija modela i hiperparametara	45
Slika 41. Report na pragu 0.5 i matrica zabune.....	46
Slika 42. Odabrani F2 prag i matrica	47
Slika 43. F2 u funkciji praga odluke	47
Slika 44. Krivulja preciznost–odziv s označenim pragovima	48
Slika 45. Važnost značajki	48
Slika 46. Helperi za inference i demo letova	49
Slika 47. Klasifikacijski izvještaj (rezultati modela).....	50
Slika 48. Matrica zabune	51
Slika 49. Klasifikacijski izvještaj za prag F2 i matrica zabune	52
Slika 50. ROC AUC i PR AUC	53
Slika 51. Važnost značajki	54
Slika 52. Rezultati predikcije triju letova.....	54